File I/O and Serialization

#	Scenario Title / Use Case	Level	Target Role
1	File content comparison ignoring whitespace	L1	Java Developer, QA Engineer
2	Size-based log rotation	L2	Backend Engineer, DevOps
3	Parse CSV rows into domain objects	L2	Java Developer, Data Engineer
4	Persist & restore game state via serialization	L1	Game Developer, Java Engineer
5	Search keyword in large file	L1	Java Developer
6	Clean outdated log entries	L2	Backend Engineer
7	Serialize accounts excluding sensitive info	L2	Security Engineer, Java Dev
8	Read binary bytes from image files	L1	Java Developer, Systems Engineer
9	Save text line-by-line safely	L1	Java Developer
10	Serialize/deserialize user info	L2	Java Developer
11	Read large binaries in chunks	L3	Backend Engineer
12	Dynamic directory & file creation	L1	Java Developer
13	Append logs to daily date-named files	L2	Backend Engineer
14	Load config options line-by-line	L1	Java Developer
15	Read numeric binary data (DataInputStream)	L3	Backend Engineer
16	Persist/load large list of strings	L2	Java Developer
17	Serialize multiple objects to one file	L3	Backend Engineer
18	Per-thread temp files with cleanup	L3	Backend Engineer
19	Manage .properties config file	L1	Java Developer
20	GZIP file compression before transfer	L3	DevOps Engineer, Backend
21	Parallel chunk processing with FileChannel/mmap	L4	Data Engineer, Performance Specialist
22	Watch multiple directories with WatchService	L3	Backend / DevOps Engineer
23	Implement fixed-size circular log file	L3	Backend Engineer
24	Thread-safe shared log writing	L3	Backend Engineer

25	Non-blocking concurrent reads (AsyncFileChannel)	L4	Senior Backend Engineer
26	Stream large CSV validation (memory-safe)	L3	Data Engineer
27	Resumable HTTP downloads with Range headers	L4	System Engineer, Backend Dev
28	Stream large JSON with Jackson tokens	L3	Backend Engineer
29	Remote HTTP tail-f via Range requests	L4	DevOps / System Engineer
30	Multi-threaded downloader with checkpoints	L4	Senior Backend / System Engineer
31	In-memory cache with periodic durable snapshots	L4	Backend Architect
32	Encrypt files at rest with AES-GCM	L3	Security Engineer
33	Safe wire serialization masking sensitive data	L3	Security Engineer
34	Random-access file regions (RAF/FileChannel.map)	L4	System Engineer
35	Stream audio in fixed-size chunks over socket	L3	Network / Java Engineer
36	Handle very large JSON via streaming	L3	Backend / Data Engineer
37	Directory watcher for create/modify/delete events	L3	DevOps Engineer
38	ZIP multiple files into a single archive	L2	Java Developer
39	Buffered logger with timed batch flush	L4	Backend Architect
40	Split large file into fixed-size transferable parts	L3	Backend / Data Engineer

Scenario 1: File content comparison ignoring spaces and line breaks

In this case, the requirement is to compare two text files to check if their content is logically identical, but without being strict about spaces or line breaks. This is useful in real-world cases like comparing configuration files, validating generated reports, or checking student-submitted assignments where formatting may differ.

Answer:

I'd approach this by normalizing both files before comparison. Normalization means removing unnecessary whitespace (spaces, tabs, newlines) so only meaningful text remains. Once normalized, we can directly compare the contents.

There are two ways:

1. Line-by-line comparison with BufferedReader, trimming whitespace.
2. Reading the entire file, stripping out all whitespace with regex, then comparing the final strings.

For smaller files, the second approach is simpler. For very large files, the line-by-line approach is more memory-efficient.

Code:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class FileComparator {

    public boolean areFilesEqual(String path1, String path2) {
        try {
            String normalized1 = normalizeFile(path1);
            String normalized2 = normalizeFile(path2);
            return normalized1.equals(normalized2);
        } catch (IOException e) {
            e.printStackTrace();
            return false;
        }
    }

    private String normalizeFile(String path) throws IOException {
        StringBuilder sb = new StringBuilder();
        try (BufferedReader br = new BufferedReader(new FileReader(path))) {
            String line;
            while ((line = br.readLine()) != null) {
```

```
                // Remove all whitespace (spaces, tabs, newlines)
                sb.append(line.replaceAll("\\s+", ""));
            }
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        FileComparator comparator = new FileComparator();
        boolean result = comparator.areFilesEqual("file1.txt", "file2.txt");
        System.out.println("Files are equal ignoring spaces: " + result);
    }
}
```

Reasoning to interviewer:

- Reading the file and stripping whitespace ensures we only compare meaningful text.
- Using StringBuilder is efficient compared to string concatenation.
- The regex \\s+ removes spaces, tabs, and newlines, so formatting differences won't matter.
- This approach works well when files are not extremely large. For very large files, I'd prefer streaming line-by-line comparison with trimming, to avoid keeping everything in memory.
- By keeping normalization in a separate method, the code is modular and reusable.

Future scope:

- Add support for case-insensitive comparison if needed (equalsIgnoreCase).
- Extend to ignore punctuation or special characters for certain use-cases (e.g., plagiarism detection).
- For very large files, use memory-mapped files with FileChannel and ByteBuffer for performance.
- Provide a report of mismatched lines instead of just true/false, which helps in debugging differences.

Scenario 2: Size-based log rotation for high-volume applications

In a busy service (think payments or ticketing), logs can explode in size. We need a rolling mechanism that keeps each file under a cap and cleanly switches to a new file without losing log lines.

Answer:

I'd maintain a single open writer and rotate when the next write would cross the limit. Rotation means: flush/close the current writer, compute a new file name (timestamp or index), and continue. I'll also make the logger thread-safe because logs are usually written by multiple threads.

Code:

```
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class RollingFileLogger {
    private final long maxBytes;
    private final Path directory;
    private final String baseName; // e.g., "app-log"
    private final DateTimeFormatter tsFmt =
DateTimeFormatter.ofPattern("yyyyMMdd-HHmmss");
    private BufferedWriter writer;
    private Path currentFile;
    private long currentSize; // bytes written to current file

    public RollingFileLogger(Path directory, String baseName, long maxBytes)
throws IOException {
        this.directory = directory;
        this.baseName = baseName;
        this.maxBytes = maxBytes;
        Files.createDirectories(directory);
        rotate(); // initialize first file
    }

    public synchronized void log(String line) throws IOException {
        // +1 for newline
        byte[] bytes = (line +
System.lineSeparator()).getBytes(StandardCharsets.UTF_8);
        if (currentSize + bytes.length > maxBytes) {
            rotate();
        }
        writer.write(line);
        writer.newLine();
        currentSize += bytes.length;
```

```
        // optional: flush on important events; otherwise rely on OS buffers
    }

    private void rotate() throws IOException {
        if (writer != null) {
            writer.flush();
            writer.close();
        }
        String stamp = LocalDateTime.now().format(tsFmt);
        currentFile = directory.resolve(baseName + "-" + stamp + ".log");
        writer = Files.newBufferedWriter(currentFile, StandardCharsets.UTF_8,
StandardOpenOption.CREATE_NEW);
        currentSize = 0L;
    }

    public synchronized void close() throws IOException {
        if (writer != null) {
            writer.flush();
            writer.close();
        }
    }

    // Simple demo
    public static void main(String[] args) throws Exception {
        RollingFileLogger logger = new RollingFileLogger(Paths.get("logs"), "app-
log", 1024 * 1024); // 1 MB
        for (int i = 0; i < 200000; i++) {
            logger.log("event#" + i + " - something happened");
        }
        logger.close();
    }
}
```

Reasoning to interviewer:

- Keeping a single open writer avoids reopen costs on every write and reduces contention.
- I check “will this write overflow?” before writing; if yes, rotate first to guarantee individual file size caps.
- Timestamped names simplify cleanup and analysis; we avoid collisions and keep chronological order.
- Synchronizing log and rotate is the simplest way to make it thread-safe. If throughput is critical, I’d buffer via a queue and a single writer thread.
- UTF-8 is explicit; size accounting uses the actual encoded byte length, not String.length().

Future scope:

- Add a max-files policy (delete oldest beyond N or based on total disk budget).
- Add time-based rotation (daily) in combination with size-based limits.
- Compress rotated files asynchronously (GZIP).
- Expose a LogHandler interface so we can swap disk with S3 or Kafka in the future.
- If allowed, wire an existing framework (Logback with SizeBasedTriggeringPolicy + TimeBasedRollingPolicy).

Scenario 3: Parsing CSV rows into domain objects

We're ingesting CSV exports (say, customer lists from CRM). Each row should become a typed Java object. CSVs often contain quoted fields and commas inside quotes, so we need to parse correctly.

Answer:

For correctness and maintainability, I prefer a battle-tested CSV parser (Apache Commons CSV/OpenCSV). If I can't add dependencies, I'll implement a small state-machine parser that handles quoted fields instead of split(","), which breaks on commas within quotes.

Code (no external libs, handles quotes and commas inside quotes):

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.*;

class Person {
    private final String name;
    private final int age;
    private final String city;

    Person(String name, int age, String city) {
        this.name = name;
        this.age = age;
        this.city = city;
    }

    @Override public String toString() {
        return name + " (" + age + "), " + city;
    }
}

public class CSVParser {

    // Parses a single CSV line into fields, supporting quotes and escaped quotes
("")
    private List<String> parseLine(String line) {
        List<String> fields = new ArrayList<>();
        StringBuilder cur = new StringBuilder();
        boolean inQuotes = false;
        for (int i = 0; i < line.length(); i++) {
            char c = line.charAt(i);
            if (inQuotes) {
                if (c == "") {
                    // Lookahead for escaped quote
                    if (i + 1 < line.length() && line.charAt(i + 1) == "") {
```

```
                    cur.append(""); // add literal quote
                    i++;            // skip the next quote
                } else {
                    inQuotes = false; // closing quote
                }
            } else {
                cur.append(c);
            }
        } else {
            if (c == ',') {
                fields.add(cur.toString());
                cur.setLength(0);
            } else if (c == "") {
                inQuotes = true;
            } else {
                cur.append(c);
            }
        }
    }
    fields.add(cur.toString());
    return fields;
  }

  public List<Person> parseCSV(String filePath, boolean hasHeader) throws
IOException {
    List<Person> people = new ArrayList<>();
    try (BufferedReader reader = new BufferedReader(new FileReader(filePath)))
{
        String line;
        boolean headerSkipped = !hasHeader;
        while ((line = reader.readLine()) != null) {
            if (!headerSkipped) { // skip header
                headerSkipped = true;
                continue;
            }
            if (line.trim().isEmpty()) continue;

            List<String> cols = parseLine(line);
            // Basic validation
            if (cols.size() < 3) {
                // ignore or log a warning; CSV row malformed
                continue;
            }
            String name = cols.get(0).trim();
            int age = Integer.parseInt(cols.get(1).trim());
            String city = cols.get(2).trim();

            people.add(new Person(name, age, city));
        }
    }
```

```
        return people;
    }

    public static void main(String[] args) throws Exception {
        CSVParser parser = new CSVParser();
        List<Person> people = parser.parseCSV("people.csv", true); // true if first row
is header
        people.forEach(System.out::println);
    }
}
```

Reasoning to interviewer:

- split(",") fails for lines like "Doe, John",31,"New York, NY". A tiny state machine handles quotes, commas inside quotes, and escaped quotes ("").
- I keep parsing and mapping concerns separate: parseLine returns tokens; the mapping block converts to domain objects with validation.
- The method supports optional headers and ignores blank/malformed lines gracefully.
- For large files, reading line-by-line keeps memory usage stable; we build objects incrementally.

Future scope:

- Switch to Apache Commons CSV for richer features (headers, schema validation, different delimiters, input streams).
- Add schema checks and typed converters (dates, enums).
- Parallelize processing with a bounded queue if lines are independent and mapping is CPU-heavy.
- Collect row-level errors to a side report for operators to fix bad inputs.

Scenario 4: Persisting and restoring a player's game state

In gaming applications, saving a player's progress (score, level, inventory) is critical. Without persistence, players would lose progress every time the app closes. A clean way in Java is to serialize the game state object to disk and reload it later.

Answer:

I'll create a GameState class that represents the player's progress. It should implement Serializable so the JVM can convert it into a byte stream. I'll then use ObjectOutputStream to save the state and ObjectInputStream to restore it later.

This approach lets us quickly persist complex objects without writing custom parsing logic.

Code:

```
import java.io.*;
import java.util.Arrays;

class GameState implements Serializable {
    private static final long serialVersionUID = 1L;

    private final int score;
    private final int level;
    private final String[] inventory;

    public GameState(int score, int level, String[] inventory) {
        this.score = score;
        this.level = level;
        this.inventory = inventory;
    }

    public int getScore() { return score; }
    public int getLevel() { return level; }
    public String[] getInventory() { return inventory; }

    @Override
    public String toString() {
        return "Score=" + score + ", Level=" + level + ", Inventory=" +
Arrays.toString(inventory);
    }
}

public class GameStateHandler {

    public void save(GameState state, String filePath) throws IOException {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(filePath))) {
```

```
            oos.writeObject(state);
        }
    }

    public GameState load(String filePath) throws IOException,
ClassNotFoundException {
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(filePath))) {
            return (GameState) ois.readObject();
        }
    }

    // Demo
    public static void main(String[] args) throws Exception {
        GameStateHandler handler = new GameStateHandler();

        GameState state1 = new GameState(1200, 5, new String[]{"Sword", "Potion",
"Shield"});
        handler.save(state1, "game.dat");

        GameState restored = handler.load("game.dat");
        System.out.println("Restored: " + restored);
    }
}
```

Reasoning to interviewer:

- Marking the class Serializable allows Java's built-in serialization to take care of persisting object graphs.
- The serialVersionUID ensures compatibility between versions of the class across different runs.
- By keeping fields private and providing getters, the class remains encapsulated while still serializable.
- ObjectOutputStream and ObjectInputStream handle deep object structures, so arrays like inventory are stored without extra code.
- The demo shows saving to game.dat and restoring it back, proving persistence works.

If I had to explain to a game designer: it's like taking a snapshot of the player's backpack and progress, freezing it, and then reloading that snapshot whenever they return.

Future scope:

- Encrypt the serialized file for security (to avoid cheating by editing the file).
- Compress the state before saving if inventory grows large.
- Consider JSON or protobuf serialization if the game needs cross-platform compatibility.
- Add versioning logic inside GameState for backward compatibility (e.g., when new fields are added).
- Store multiple checkpoints (auto-save, manual save) in separate files with timestamps.

Scenario 5: Searching for lines with a keyword in a large file

Imagine analyzing a huge server log where you want all entries containing “ERROR”. Reading everything into memory at once is inefficient. The right way is to stream through the file line by line, check for the keyword, and collect matches.

Answer:

I’ll use a BufferedReader for efficient line-by-line reading. As I read, I’ll check if the line contains the keyword. If it does, I’ll store it in a list. Finally, return all matches. This keeps memory usage under control because we only hold the matches, not the whole file.

Code:

```
import java.io.*;
import java.util.*;

public class KeywordFinder {

    public List<String> findLinesWithKeyword(String filePath, String keyword)
throws IOException {
        List<String> matches = new ArrayList<>();
        try (BufferedReader reader = new BufferedReader(new FileReader(filePath)))
{
            String line;
            while ((line = reader.readLine()) != null) {
                if (line.contains(keyword)) {
                    matches.add(line);
                }
            }
        }
        return matches;
    }

    public static void main(String[] args) throws Exception {
        KeywordFinder finder = new KeywordFinder();
        List<String> results = finder.findLinesWithKeyword("application.log",
"ERROR");
        results.forEach(System.out::println);
    }
}
```

Reasoning to interviewer:

- BufferedReader is efficient for large files because it buffers disk reads.

- Using line.contains(keyword) is simple and works for case-sensitive matching.
- Only matched lines are added to the list, so memory use scales with the number of matches, not the whole file size.
- The method is reusable for any keyword, not tied to a specific one.

Future scope:

- Add case-insensitive search using toLowerCase().
- Support regex-based matching for advanced searches (e.g., IP addresses).
- Stream results to an output file instead of holding them in memory for extremely large datasets.
- Parallelize across chunks of the file with Files.lines() and streams for performance on multi-core systems.

Scenario 6: Cleaning outdated records from a log file

Suppose we maintain a transaction log, but only want to keep records from the last 30 days. We need to scan the log, keep valid lines, and remove old ones.

Answer:

Since we can't directly delete lines from the middle of a file, the typical way is to filter into a temporary structure, then overwrite the file with the filtered content. I'll read the file line by line, apply the condition, and rewrite only the valid ones back.

Code:

```
import java.io.*;
import java.util.*;

public class LogCleaner {

    // Keeps only lines that MATCH the filter condition
    public void retainMatchingRecords(String filePath, String filter) throws
IOException {
        List<String> filteredLines = new ArrayList<>();

        // Read and filter
        try (BufferedReader reader = new BufferedReader(new FileReader(filePath)))
{
            String line;
            while ((line = reader.readLine()) != null) {
                if (line.contains(filter)) {
                    filteredLines.add(line);
                }
            }
        }

        // Rewrite file with filtered content
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath))) {
            for (String line : filteredLines) {
                writer.write(line);
                writer.newLine();
            }
        }
    }

    public static void main(String[] args) throws Exception {
        LogCleaner cleaner = new LogCleaner();
        cleaner.retainMatchingRecords("server.log", "2025-09"); // keep only
September logs
    }
```

```
}
```

Reasoning to interviewer:

- Text files don’t support in-place removal of arbitrary lines, so rewriting is the practical solution.
- This implementation is simple and works well for moderately sized files.
- I separated the reading (filtering) and writing (overwriting) phases to make the logic clear.
- The filter condition is flexible: it could be a date string, keyword, or regex.

Future scope:

- Use a temporary file during writing, then replace the old file to avoid corruption if a crash happens mid-write.
- Support more advanced filters (by date, log severity, user ID).
- For very large files, process in streaming mode: read each line and immediately write back valid ones without keeping all in memory.
- Integrate with Java NIO Files.lines() and Stream.filter() for more declarative code.

Scenario 7: Serializing user accounts while excluding sensitive fields

In many applications (like banking or e-commerce), we need to persist user accounts but must never expose sensitive data like passwords or tokens. Java gives us a way to explicitly skip such fields during serialization.

Answer:

The cleanest approach is to mark sensitive fields with the transient keyword. When the JVM serializes the object, transient fields are ignored. At deserialization time, they are restored with default values (null for objects, 0 for primitives).

This way, the account ID and other details are safely serialized, but the password is left out.

Code:

```
import java.io.*;

class Account implements Serializable {
    private static final long serialVersionUID = 1L;

    private final String accountId;
    private transient String password; // excluded from serialization

    public Account(String accountId, String password) {
        this.accountId = accountId;
        this.password = password;
    }

    public String getAccountId() { return accountId; }
    public String getPassword() { return password; }

    @Override
    public String toString() {
        return "Account{id=" + accountId + ", password=" + password + "}";
    }
}

public class AccountSerializationDemo {
    public static void main(String[] args) throws Exception {
        Account acc = new Account("U123", "secret123");

        // Serialize
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("acc.dat"))) {
            oos.writeObject(acc);
        }
```

```
        // Deserialize
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("acc.dat"))) {
            Account restored = (Account) ois.readObject();
            System.out.println("Restored: " + restored); // password will be null
        }
    }
}
```

Reasoning to interviewer:

- transient ensures sensitive data never leaves memory during serialization.
- After deserialization, the password is reset to null, protecting against leaks.
- If needed, we could reinitialize such fields after loading (e.g., fetching password hash from a secure vault).
- This is far safer than serializing everything by default.

Future scope:

- For advanced control, implement writeObject and readObject methods to encrypt or mask sensitive fields.
- Use external serialization formats like JSON/Protobuf where you can control field inclusion.
- Integrate with java.security libraries if sensitive fields must be encrypted rather than excluded.

Scenario 8: Reading raw binary data from image files

Sometimes, we need to process an image not as pixels but as raw bytes (e.g., uploading to a server, computing checksums, or encrypting before storage). Text-based readers won't work here because image data is binary.

Answer:

I'll use a FileInputStream, which reads raw bytes directly. We can either read into a pre-sized byte array or read in chunks for large files. This avoids character encoding issues since we're treating the file as pure binary.

Code:

```
import java.io.*;
import java.nio.file.*;

public class BinaryImageReader {

    // Reads the entire image into a byte array
    public byte[] readImage(String imagePath) throws IOException {
        File file = new File(imagePath);
        try (FileInputStream fis = new FileInputStream(file)) {
            byte[] data = new byte[(int) file.length()];
            int read = fis.read(data);
            if (read != data.length) {
                throw new IOException("Could not read the entire file");
            }
            return data;
        }
    }

    // Alternative modern approach with NIO
    public byte[] readImageNio(String imagePath) throws IOException {
        return Files.readAllBytes(Paths.get(imagePath));
    }

    public static void main(String[] args) throws Exception {
        BinaryImageReader reader = new BinaryImageReader();
        byte[] bytes = reader.readImage("photo.jpg");
        System.out.println("Read " + bytes.length + " bytes.");
    }
}
```

Reasoning to interviewer:

- FileInputStream reads at the byte level, so no encoding or conversion corrupts binary data.
- Using file.length() ensures we allocate an array of exact size.

- The NIO Files.readAllBytes() is simpler and handles the details internally.
- With this, we can later manipulate the bytes (resize, hash, compress, encrypt).

Future scope:

- For very large images, read in buffered chunks (byte[4096]) to avoid huge memory allocations.
- Wrap FileInputStream in BufferedInputStream for better I/O performance.
- If further image processing is needed (pixels, metadata), integrate with ImageIO on top of the raw byte read.

Scenario 9: Saving text input line-by-line with proper exception handling

Think of a simple text editor like Notepad. When the user presses Save, the program must persist the content exactly as typed — preserving line breaks — and handle disk or permission errors gracefully.

Answer:

I'd use BufferedWriter inside a try-with-resources block. BufferedWriter batches writes, which improves performance compared to writing character by character. The try-with-resources ensures the writer is closed automatically even if an exception occurs.

If the input content comes as a multi-line string, I'll split it by line separators and write each line individually.

Code:

```
import java.io.*;
import java.util.List;

public class TextFileSaver {

    public void saveTextContent(List<String> lines, String targetFile) {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(targetFile))) {
            for (String line : lines) {
                bw.write(line);
                bw.newLine();  // ensures correct line breaks across platforms
            }
            System.out.println("File saved successfully.");
        } catch (IOException ex) {
            System.err.println("Failed to save content: " + ex.getMessage());
        }
    }

    public static void main(String[] args) {
        TextFileSaver saver = new TextFileSaver();
        saver.saveTextContent(
                List.of("Hello, World!", "This is line two.", "End of file."),
                "notes.txt"
        );
    }
}
```

Reasoning to interviewer:

- BufferedWriter avoids too many system calls by buffering text before writing.

- Writing each line followed by newLine() preserves formatting, regardless of the underlying OS.
- try-with-resources guarantees resource cleanup and avoids memory leaks.
- Separating each line write makes the code more adaptable (e.g., handling auto-save features line-by-line).

Future scope:

- Add support for appending instead of overwriting (use FileWriter(file, true)).
- Auto-save at regular intervals using a scheduler.
- For very large files, consider Files.newBufferedWriter from NIO for more control over encodings.
- Provide user feedback through a UI message or logs if saving fails.

Scenario 10: Serializing and deserializing user information in binary format

Let's say we're building a user management system for a desktop app. Instead of saving users to a text file, we want to persist them in binary form and restore them later into User objects.

Answer:

The simplest way is to make the User class implement Serializable. Serialization turns the object into a byte stream that can be written to a file with ObjectOutputStream. Later, ObjectInputStream reconstructs it back into a Java object.

Code:

```
import java.io.*;

class User implements Serializable {
    private static final long serialVersionUID = 1L;

    private final int userId;
    private final String userName;
    private final String userEmail;

    public User(int userId, String userName, String userEmail) {
        this.userId = userId;
        this.userName = userName;
        this.userEmail = userEmail;
    }

    @Override
    public String toString() {
        return "User{id=" + userId + ", name='" + userName + "', email='" + userEmail
+ "'}";
    }

    public static void serializeUser(User user, String path) throws IOException {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(path))) {
            oos.writeObject(user);
            System.out.println("User serialized successfully.");
        }
    }

    public static User deserializeUser(String path) throws IOException,
ClassNotFoundException {
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(path))) {
            return (User) ois.readObject();
```

```
        }
    }

    public static void main(String[] args) throws Exception {
        User u1 = new User(101, "Alice", "alice@example.com");
        serializeUser(u1, "user.dat");

        User restored = deserializeUser("user.dat");
        System.out.println("Restored: " + restored);
    }
}
```

Reasoning to interviewer:

- Implementing Serializable allows Java's native serialization mechanism to handle object graphs.
- The serialVersionUID ensures class compatibility across versions.
- Using try-with-resources guarantees streams are closed, preventing file locks or leaks.
- This approach works well for quick persistence in desktop or small applications.

Future scope:

- For sensitive fields (like passwords), use transient to exclude them.
- For cross-language compatibility (say, Java + Python), use formats like JSON or Protocol Buffers instead of native serialization.
- Store multiple users in a list and serialize the list instead of single objects.
- Add versioning support if the schema evolves (extra fields in future releases).

Scenario 11: Reading large binary files in manageable chunks

When dealing with very large files, say multi-GB video or data dumps, loading the entire file into memory is risky and inefficient. Instead, we should stream it in smaller byte chunks, process each chunk, and move on.

Answer:

A good approach is to use BufferedInputStream wrapped around a FileInputStream. This lets us repeatedly fill a byte array buffer (say 1 KB, 4 KB, or 8 KB at a time) until the end of the file. This avoids memory overload and keeps I/O efficient.

Code:

```
import java.io.BufferedInputStream;
import java.io.FileInputStream;
import java.io.IOException;

public class BinaryChunkReader {

    public void readFileInChunks(String filePath) {
        try (BufferedInputStream bis = new BufferedInputStream(new
FileInputStream(filePath))) {
            byte[] buffer = new byte[4096]; // 4 KB buffer
            int bytesRead;
            while ((bytesRead = bis.read(buffer)) != -1) {
                // Process only buffer[0..bytesRead-1]
                System.out.println("Read " + bytesRead + " bytes.");
            }
        } catch (IOException e) {
            System.err.println("Error reading file: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        new BinaryChunkReader().readFileInChunks("largefile.bin");
    }
}
```

Reasoning to interviewer:

- Streaming with a buffer prevents memory exhaustion on huge files.
- BufferedInputStream reduces system calls by batching underlying reads.
- Using bytesRead ensures we only process valid bytes in the buffer (the last read may not fill the buffer completely).
- The buffer size is tunable — larger buffers improve throughput, smaller ones save memory.

Future scope:

- Make the buffer size configurable based on file size or system memory.
- Use FileChannel + ByteBuffer (NIO) for more control and possible memory-mapped optimizations.
- Parallelize processing if the file format allows independent chunk operations.
- Integrate checksums (MD5/SHA) while streaming for data validation.

Scenario 12: Dynamic directory and file creation

Suppose users upload documents to paths like /data/users/2025/january/. Before saving the file, we must ensure all directories exist, otherwise the write will fail.

Answer:

We can use the File class (or Path in NIO). The mkdirs() method creates all missing parent directories. Once the directory exists, we call createNewFile() on the file. This guarantees the full path structure is in place before writing.

Code:

```
import java.io.File;
import java.io.IOException;

public class DirectoryFileCreator {

    public void setupFile(String dirPath, String fileName) {
        File directory = new File(dirPath);
        if (!directory.exists()) {
            if (directory.mkdirs()) {
                System.out.println("Created directory: " + dirPath);
            } else {
                System.out.println("Failed to create directory: " + dirPath);
                return;
            }
        }

        File file = new File(directory, fileName);
        try {
            if (file.createNewFile()) {
                System.out.println("File created at: " + file.getAbsolutePath());
            } else {
                System.out.println("File already exists: " + file.getAbsolutePath());
            }
        } catch (IOException e) {
            System.err.println("Error creating file: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        new DirectoryFileCreator().setupFile("data/users/2025/january", "report.txt");
    }
}
```

Reasoning to interviewer:

- mkdirs() ensures the entire directory chain is created (not just the last folder).

- createNewFile() is safe: it returns false if the file already exists, avoiding accidental overwrites.
- By separating directory and file creation, we avoid errors where a file is attempted in a non-existent directory.
- Using File makes it easy to manage paths without manual string concatenation.

Future scope:

- Switch to NIO (Files.createDirectories(Paths.get(dirPath))) for better exception handling and modern APIs.
- Add file permissions setup (read-only, owner-only).
- Extend logic to create temporary files with File.createTempFile().
- Add logging or alerts if directory creation fails (e.g., insufficient permissions).

Scenario 13: Appending logs to daily files named by date

In a production system, logs are often rotated daily. We want all events of a single day to land in one file (like log_2025-09-14.txt), and new entries should be appended, not overwrite what's already there.

Answer:

The solution is to generate the file name using the current date and open the file in append mode. In Java, FileWriter provides a constructor flag that ensures data is added to the file instead of replacing it. For better performance, I'll wrap it with a BufferedWriter.

Code:

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;

public class DateBasedLogger {

    public void logMessage(String message) {
        String logFileName = "log_" + LocalDate.now() + ".txt";
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(logFileName,
true))) {
            bw.write(message);
            bw.newLine();
            System.out.println("Log entry added to " + logFileName);
        } catch (IOException e) {
            System.err.println("Error writing log: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        DateBasedLogger logger = new DateBasedLogger();
        logger.logMessage("Application started.");
        logger.logMessage("Processing user request...");
    }
}
```

Reasoning to interviewer:

- LocalDate.now() provides a natural file name keyed by date.
- Using append mode (true) ensures existing logs are preserved.
- Wrapping FileWriter in BufferedWriter improves efficiency by reducing disk writes.

- This design keeps logs separated per day, making them easier to archive, search, or delete.

Future scope:

- Add timestamps per line (LocalDateTime.now()) for better tracking.
- Implement log rotation by size as well as date (e.g., log_2025-09-14-1.txt).
- Integrate with asynchronous logging (queue + worker thread) to avoid blocking main threads.
- Support log levels (INFO, ERROR) and direct them to different files.

Scenario 14: Loading configuration options line-by-line

At startup, most apps read a config file that stores settings like DB URLs, API keys, or feature toggles. Each line may represent a single property.

Answer:

I'd use BufferedReader to stream through the file. Each line can be parsed into key-value pairs (if using a key=value format), or simply processed as plain strings. This is efficient for small-to-medium config files, which are typically just a few KB.

Code:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class ConfigLoader {

    public void loadConfig(String filePath) {
        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
            String line;
            while ((line = br.readLine()) != null) {
                line = line.trim();
                if (line.isEmpty() || line.startsWith("#")) {
                    continue; // skip blank lines and comments
                }
                System.out.println("Setting: " + line);
            }
        } catch (IOException e) {
            System.err.println("Error reading config: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        ConfigLoader loader = new ConfigLoader();
        loader.loadConfig("app.config");
    }
}
```

Reasoning to interviewer:

- BufferedReader is efficient for sequential reads.
- Trimming removes trailing/leading spaces.
- Skipping blank lines and comments (#) makes it more robust.
- Printing or parsing each line gives flexibility — we can later map it into a Map<String, String>.

Future scope:

- Parse lines into key=value and store them in a Properties object for easy lookup.
- Switch to java.util.Properties.load() if the file follows the properties format.
- Add validation (e.g., required keys must exist).
- Hot-reload the config file with a watcher so changes apply without restarting the app.

Scenario 15: Reading binary numbers from large binary files

Suppose we're dealing with a binary data dump — for example, sensor readings from an IoT device or packed financial data. Each number isn't just text, but raw bytes representing integers, doubles, or floats. We need a way to read them correctly, respecting byte boundaries.

Answer:

The right tool here is DataInputStream, which wraps around a FileInputStream. It provides type-safe methods like readInt(), readDouble(), readLong(), etc., so we don't have to manually interpret byte arrays. This makes reading structured numeric binary data straightforward.

Code:

```
import java.io.DataInputStream;
import java.io.FileInputStream;
import java.io.IOException;

public class NumericBinaryReader {

    public void readNumbers(String filePath) {
        try (DataInputStream dis = new DataInputStream(new
FileInputStream(filePath))) {
            while (dis.available() > 0) {
                int num = dis.readInt();   // 4 bytes read as integer
                System.out.println("Read number: " + num);
            }
        } catch (IOException e) {
            System.err.println("Error reading numbers: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        new NumericBinaryReader().readNumbers("numbers.bin");
    }
}
```

Reasoning to interviewer:

- DataInputStream abstracts byte parsing, so I don't have to manually handle endianness for basic types.
- The available() check ensures we don't read beyond EOF.
- This method assumes the file was written with matching methods like DataOutputStream.writeInt().

- The approach is both efficient and less error-prone compared to manual byte shifting.

Future scope:

- Handle different numeric types (floats, doubles, longs).
- If data comes in a custom binary format (e.g., little-endian), use ByteBuffer with order(ByteOrder.LITTLE_ENDIAN).
- Support streaming large files in chunks if values are very large arrays.
- Add schema awareness (field definitions) for structured binary records.

Scenario 16: Saving and loading large lists of strings

Think of a chat application or a dictionary app that needs to persist thousands or millions of strings (messages, words). Each string should be stored on a separate line so it can be reconstructed easily.

Answer:

I'll use BufferedWriter to efficiently write each string line-by-line. To load them back, BufferedReader will read the file line-by-line into a list. This streaming approach avoids memory overload, since it doesn't require loading the whole file at once.

Code:

```
import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class StringListFileHandler {

    public void writeStrings(List<String> list, String filePath) throws IOException {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(filePath))) {
            for (String s : list) {
                bw.write(s);
                bw.newLine();
            }
        }
    }

    public List<String> readStrings(String filePath) throws IOException {
        List<String> list = new ArrayList<>();
        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
            String line;
            while ((line = br.readLine()) != null) {
                list.add(line);
            }
        }
        return list;
    }

    public static void main(String[] args) throws Exception {
        StringListFileHandler handler = new StringListFileHandler();

        // Writing
        List<String> words = List.of("apple", "banana", "cherry");
        handler.writeStrings(words, "words.txt");

        // Reading back
        List<String> restored = handler.readStrings("words.txt");
        restored.forEach(System.out::println);
```

```
    }
}
```

Reasoning to interviewer:

- Writing with BufferedWriter ensures fewer disk writes, which matters when storing millions of lines.
- Reading line-by-line is memory-friendly because we process one line at a time.
- The design cleanly separates responsibilities: writeStrings for saving, readStrings for restoring.
- This pattern works well for log files, chat transcripts, or datasets like dictionaries.

Future scope:

- For extremely large datasets, stream results to the caller instead of building a full list in memory.
- Use Files.lines(Path) from NIO for lazy streaming.
- Handle encodings explicitly (UTF-8) to avoid platform-specific issues.
- Add compression (e.g., GZIP) if file size is a concern.

Scenario 17: Serializing many objects into a single file and restoring them one by one

Think of exporting a batch of orders at day end. You want to dump each Order object to one file and later read them back sequentially for reconciliation.

Answer:

I'll open one ObjectOutputStream and call writeObject for each object in sequence. To read, I'll open an ObjectInputStream and call readObject in a loop until I hit EOFException, which is the natural end-of-stream signal. If I need to append in multiple sessions, I'll avoid writing multiple stream headers by using a header-skipping ObjectOutputStream subclass.

Code:

```
import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class MultiObjectIO {

    // Use when writing everything in a single session
    public static void writeAll(List<? extends Serializable> items, String file) throws
IOException {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(file))) {
            for (Serializable it : items) {
                oos.writeObject(it);
            }
        }
    }

    // Use when appending objects across multiple sessions (avoid extra stream
headers)
    static class AppendingObjectOutputStream extends ObjectOutputStream {
        AppendingObjectOutputStream(OutputStream out) throws IOException {
super(out); }
        @Override protected void writeStreamHeader() throws IOException { reset();
} // skip header
    }

    public static void appendAll(List<? extends Serializable> items, String file)
throws IOException {
        boolean exists = new File(file).exists();
        try (ObjectOutputStream oos = exists
                ? new AppendingObjectOutputStream(new FileOutputStream(file, true))
                : new ObjectOutputStream(new FileOutputStream(file))) {
```

```
            for (Serializable it : items) {
                oos.writeObject(it);
            }
        }
    }

    public static <T> List<T> readAll(String file) throws IOException,
ClassNotFoundException {
        List<T> out = new ArrayList<>();
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(file))) {
            while (true) {
                try {
                    @SuppressWarnings("unchecked")
                    T obj = (T) ois.readObject();
                    out.add(obj);
                } catch (EOFException end) {
                    break; // reached end of file
                }
            }
        }
        return out;
    }

    // tiny demo with Strings (Serializable)
    public static void main(String[] args) throws Exception {
        String path = "objects.bin";
        writeAll(List.of("A", "B", "C"), path);
        appendAll(List.of("D", "E"), path);

        List<String> restored = readAll(path);
        System.out.println(restored); // [A, B, C, D, E]
    }
}
```

Reasoning to interviewer:

- EOFException is the idiomatic way to detect the end when reading a sequence of serialized objects.
- Writing multiple sessions to the same file requires skipping the stream header on subsequent appends; otherwise ObjectInputStream will choke.
- The generic readAll returns typed results with a single cast point.
- Using try-with-resources guarantees streams close on success or failure.

Future scope:

• Add per-record checksums or version fields to detect corruption or schema changes.
• Consider a container format (length-prefixed frames) to allow random access and parallel reads.
• For cross-language needs, switch to protobuf/Avro rather than native Java serialization.

Scenario 18: Per-thread temporary files that get cleaned up after work completes

Picture a parallel image pipeline where each worker thread writes intermediate bytes to a temp file. We want unique names and reliable cleanup.

Answer:

I'll use Files.createTempFile to create a unique file for each thread. For long-running services, I won't rely only on deleteOnExit() (cleanup happens only at JVM shutdown). Instead, I'll delete the file in a finally block when the thread finishes. If I still want a safety net, I can also call deleteOnExit().

Code:

```
import java.io.IOException;
import java.nio.file.*;
import java.util.concurrent.*;

public class TempFileManager {

    static class Worker implements Callable<Path> {
        @Override
        public Path call() throws Exception {
            Path temp = Files.createTempFile("worker-", ".tmp"); // unique, secure
            try {
                // ... write intermediate data to 'temp' ...
                // Files.writeString(temp, "intermediate-
"+Thread.currentThread().getName());
                return temp; // maybe hand off path for post-processing
            } finally {
                // If the file is no longer needed by the caller, delete here:
                try {
                    Files.deleteIfExists(temp);
                } catch (IOException ignored) {}
            }
        }
    }

    public static void main(String[] args) throws Exception {
        ExecutorService pool = Executors.newFixedThreadPool(3);
        try {
            Future<Path> f1 = pool.submit(new Worker());
            Future<Path> f2 = pool.submit(new Worker());
            Future<Path> f3 = pool.submit(new Worker());

            // If a caller needs to inspect files before deletion,
            // move deletion responsibility outside the worker.
            f1.get(); f2.get(); f3.get();
        } finally {
```

```
            pool.shutdown();
        }
    }
}
```

Reasoning to interviewer:

- Files.createTempFile guarantees unique names and avoids predictable filename issues.
- Deleting in finally makes cleanup deterministic per thread; deleteOnExit() alone isn't ideal for services that run for days.
- If downstream code needs the temp file after the worker finishes, return the Path and shift deletion to the consumer once it's done.
- Using an ExecutorService mirrors real production setups where multiple workers run in parallel.

Future scope:

- Use Files.createTempDirectory per thread for multiple scratch files, then delete the whole directory tree on completion.
- On POSIX systems, set restrictive permissions (owner-only) with PosixFilePermissions.
- If files are large, consider memory-mapped I/O for performance and delete via Files.delete after closing channels.
- Add a watchdog that periodically purges stale temp files in case of unexpected crashes.

Scenario 19: Managing application settings with a .properties file

Most Java apps use .properties files for configuration—think database URLs, API keys, or feature flags. They're human-readable, easy to edit, and Java provides native support.

Answer:

I'll use the Properties class, which is purpose-built for key-value settings. To save, I'll call store(), and to read, I'll call load(). The API works seamlessly with input/output streams and handles the standard .properties syntax.

Code:

```
import java.io.*;
import java.util.Properties;

public class ConfigManager {

    // Save or update a key-value pair
    public void saveConfig(String filePath, String key, String value) throws
IOException {
        Properties props = new Properties();

        // If file exists, load existing properties so we don't overwrite them
        File file = new File(filePath);
        if (file.exists()) {
            try (FileInputStream in = new FileInputStream(file)) {
                props.load(in);
            }
        }

        props.setProperty(key, value);

        try (FileOutputStream out = new FileOutputStream(file)) {
            props.store(out, "Application Configurations");
        }
    }

    // Load a specific property
    public String loadConfig(String filePath, String key) throws IOException {
        Properties props = new Properties();
        try (FileInputStream in = new FileInputStream(filePath)) {
            props.load(in);
        }
        return props.getProperty(key);
    }

    // Demo
    public static void main(String[] args) throws Exception {
```

```
        ConfigManager cm = new ConfigManager();
        cm.saveConfig("app.properties", "db.url", "jdbc:mysql://localhost:3306/mydb");
        cm.saveConfig("app.properties", "db.user", "admin");

        String url = cm.loadConfig("app.properties", "db.url");
        System.out.println("Loaded db.url = " + url);
    }
}
```

Reasoning to interviewer:

- Properties handles both saving and loading key-value pairs.
- Before saving, I load existing values so new writes don't wipe out old settings.
- Using try-with-resources ensures streams close even if errors occur.
- This approach works for both small config files and larger ones with dozens of entries.

Future scope:

- Add validation for required keys (fail fast if db.url is missing).
- Support default values by chaining getProperty(key, default).
- Migrate to YAML/JSON for complex structured configs (nested sections).
- Add live reload by watching the properties file with WatchService.

Scenario 20: Compressing files before transfer with GZIP

In apps where users send files to each other (say chat apps or cloud storage), compressing before transfer saves bandwidth and reduces latency.

Answer:

Java provides GZIPOutputStream for compression. I'll read the original file in chunks with a FileInputStream, then pipe those chunks into a GZIPOutputStream. The result is a smaller .gz file that can be decompressed later with GZIPInputStream.

Code:

```
import java.io.*;
import java.util.zip.GZIPOutputStream;

public class FileCompressor {

    public void compressFile(String originalFilePath, String compressedFilePath)
throws IOException {
        try (FileInputStream fis = new FileInputStream(originalFilePath);
             FileOutputStream fos = new FileOutputStream(compressedFilePath);
             GZIPOutputStream gzipOut = new GZIPOutputStream(fos)) {

            byte[] buffer = new byte[4096]; // 4 KB buffer
            int bytesRead;
            while ((bytesRead = fis.read(buffer)) != -1) {
                gzipOut.write(buffer, 0, bytesRead);
            }

            System.out.println("File compressed to: " + compressedFilePath);
        }
    }

    public static void main(String[] args) throws Exception {
        FileCompressor compressor = new FileCompressor();
        compressor.compressFile("report.txt", "report.txt.gz");
    }
}
```

Reasoning to interviewer:

- Reading in chunks prevents memory overload on large files.
- GZIPOutputStream automatically handles compression using the GZIP algorithm.
- Closing the stream with try-with-resources ensures the GZIP footer is written correctly.

- This approach is simple and integrates directly into file-transfer pipelines.

Future scope:

- Add a matching decompression method using GZIPInputStream.
- Compress multiple files into a single ZIP archive (ZipOutputStream) instead of individual .gz files.
- Add progress reporting (e.g., percentage complete) for large files.
- Parallelize compression for very large files using multiple threads and chunking.

Scenario 21: Concurrently processing a very large numeric file by splitting into chunks

We're parsing a giant numeric dump (think telemetry or market data). One core would be slow, so we'll split the file into segments and let a pool of threads process their own regions in parallel.

Answer:

I'll use FileChannel + memory mapping. For parallelism, each thread maps only its chunk (map(READ_ONLY, offset, length)) and parses locally. This avoids extra copying and lets the OS page in data on demand. If records are newline-delimited, I'll align chunk boundaries so we don't split numbers across lines (give each chunk a small overlap and fix boundaries at the next newline).

Code:

```
import java.io.IOException;
import java.io.RandomAccessFile;
import java.nio.MappedByteBuffer;
import java.nio.channels.FileChannel;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;

public class ParallelNumericFileProcessor {

    static class ChunkTask implements Callable<Long> {
        private final MappedByteBuffer buf; // this chunk only (<= 2GB)

        ChunkTask(MappedByteBuffer buf) { this.buf = buf; }

        @Override
        public Long call() {
            long sum = 0; // example aggregation
            StringBuilder token = new StringBuilder();

            for (int i = 0; i < buf.capacity(); i++) {
                byte b = buf.get(i);
                if (b == '\n' || b == '\r') {
                    if (token.length() > 0) {
                        sum += Long.parseLong(token.toString());
                        token.setLength(0);
                    }
                } else {
                    token.append((char) b); // ASCII/UTF-8 digits
                }
            }
            if (token.length() > 0) sum += Long.parseLong(token.toString());
```

```
            return sum;
        }
    }

    public long process(String path, int threads) throws IOException,
InterruptedException, ExecutionException {
        try (RandomAccessFile raf = new RandomAccessFile(path, "r");
             FileChannel ch = raf.getChannel()) {

            long size = ch.size();
            threads = Math.max(1, Math.min(threads,
Runtime.getRuntime().availableProcessors()));
            long approxChunk = Math.max(1, size / threads);

            // First pass: compute chunk [start, end) boundaries at newlines to avoid
splitting numbers
            List<long[]> ranges = new ArrayList<>();
            long start = 0;
            while (start < size) {
                long tentativeEnd = Math.min(size, start + approxChunk);
                long end = seekNextNewline(ch, tentativeEnd, size); // move to next '\n'
if possible
                if (end <= start) end = Math.min(size, start + approxChunk); // fallback
                ranges.add(new long[]{start, end});
                start = end;
            }

            ExecutorService pool = Executors.newFixedThreadPool(threads);
            List<Future<Long>> futures = new ArrayList<>();

            for (long[] r : ranges) {
                long off = r[0], len = r[1] - r[0];
                // map a sub-2GB window; if len > Integer.MAX_VALUE, further
subdivide (not shown for brevity)
                if (len > Integer.MAX_VALUE) {
                    // subdivide huge chunk into <=2GB slices
                    long remaining = len;
                    long subStart = off;
                    while (remaining > 0) {
                        int cap = (int)Math.min(remaining, (long)Integer.MAX_VALUE);
                        MappedByteBuffer mb =
ch.map(FileChannel.MapMode.READ_ONLY, subStart, cap);
                        futures.add(pool.submit(new ChunkTask(mb)));
                        subStart += cap;
                        remaining -= cap;
                    }
                } else {
                    MappedByteBuffer mb =
ch.map(FileChannel.MapMode.READ_ONLY, off, len);
                    futures.add(pool.submit(new ChunkTask(mb)));
```

```
            }
        }

        long total = 0;
        for (Future<Long> f : futures) total += f.get();
        pool.shutdown();
        return total;
    }
  }

  // Seek forward to next newline to avoid splitting records
  private long seekNextNewline(FileChannel ch, long pos, long size) throws
IOException {
    if (pos >= size) return size;
    MappedByteBuffer mb = ch.map(FileChannel.MapMode.READ_ONLY, pos,
Math.min(1024 * 64, size - pos));
    for (int i = 0; i < mb.capacity(); i++) {
      byte b = mb.get(i);
      if (b == '\n') return pos + i + 1;
    }
    return Math.min(size, pos + mb.capacity()); // fallback within window
  }

  public static void main(String[] args) throws Exception {
    long sum = new ParallelNumericFileProcessor().process("numbers.txt", 4);
    System.out.println("Sum = " + sum);
  }
}
```

Reasoning to interviewer:

- Memory mapping lets the OS handle paging, so threads read their regions without user-space copies.
- Aligning on newline boundaries avoids splitting tokens; otherwise you'd need handoff logic for partial tokens at edges.
- The Integer.MAX_VALUE limit on MappedByteBuffer capacity is handled by subdividing large ranges.
- ExecutorService gives clean lifecycle control and scales with CPU cores.

Future scope:

- If numbers are binary (not text), parse with ByteBuffer and set order(ByteOrder.LITTLE_ENDIAN) as needed.
- Add per-chunk partial token carry-over logic if exact newline alignment isn't possible.

• Use a ring buffer or ForkJoinPool for advanced pipelining (I/O vs CPU parsing).

• Expose back-pressure and progress reporting.

Scenario 22: Watching multiple directories and reacting to file changes in real time

We're building an ingest pipeline that must react when files appear in several inbound folders (e.g., /invoices, /reports). We need a single loop that watches them all.

Answer:

I'll use the WatchService API. Each directory is registered for create/modify/delete events. I'll keep a Map<WatchKey, Path> so I know which directory an event belongs to. The watcher blocks on take(), processes events, and resets the key. If a key becomes invalid (directory deleted), I'll remove it.

Code:

```
import java.io.IOException;
import java.nio.file.*;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

import static java.nio.file.StandardWatchEventKinds.*;

public class MultiDirectoryWatcher {

    public void monitorDirectories(List<Path> dirs) throws IOException,
InterruptedException {
        try (WatchService watcher = FileSystems.getDefault().newWatchService()) {
            Map<WatchKey, Path> keyToDir = new HashMap<>();

            for (Path dir : dirs) {
                Files.createDirectories(dir); // ensure exists
                WatchKey key = dir.register(watcher, ENTRY_CREATE,
ENTRY_MODIFY, ENTRY_DELETE);
                keyToDir.put(key, dir);
            }

            while (true) {
                WatchKey key = watcher.take(); // blocks
                Path dir = keyToDir.get(key);
                if (dir == null) {
                    key.reset();
                    continue;
                }

                for (WatchEvent<?> event : key.pollEvents()) {
                    WatchEvent.Kind<?> kind = event.kind();
                    if (kind == StandardWatchEventKinds.OVERFLOW) continue;
```

```
                @SuppressWarnings("unchecked")
                WatchEvent<Path> ev = (WatchEvent<Path>) event;
                Path name = ev.context();
                Path fullPath = dir.resolve(name);

                System.out.printf("[%s] %s%n", kind.name(), fullPath);

                // react: enqueue file, reload config, etc.
                if (kind == ENTRY_CREATE) {
                    // handle new file
                } else if (kind == ENTRY_MODIFY) {
                    // handle changes
                } else if (kind == ENTRY_DELETE) {
                    // handle deletion
                }
            }

            boolean valid = key.reset();
            if (!valid) {
                keyToDir.remove(key);
                if (keyToDir.isEmpty()) break; // nothing left to watch
            }
        }
    }

    public static void main(String[] args) throws Exception {
        new MultiDirectoryWatcher().monitorDirectories(
                List.of(Paths.get("invoices"), Paths.get("reports"), Paths.get("uploads"))
        );
    }
}
```

Reasoning to interviewer:

- WatchService is efficient; it uses native OS notifications where available.
- Mapping WatchKey back to its Path disambiguates which directory the event came from.
- Handling OVERFLOW is important when events arrive faster than we can process.
- Resetting the key readies it for future events; if reset fails, that directory is no longer watchable.

Future scope:

• Debounce rapid MODIFY bursts (write temp → move final) before acting.
• Watch subdirectories recursively by registering new dirs when they’re created.
• Offload processing to a worker pool to keep the watcher loop responsive.
• Add graceful shutdown via an interrupt or a poison event.

Scenario 23: Circular log file that overwrites old entries once full

In audit or telemetry systems, storage is limited but you must keep the *latest* activity. The requirement is not to rotate files, but to keep a fixed-size file that behaves like a ring buffer.

Answer:

I'd use RandomAccessFile because it allows me to seek to any byte offset and overwrite in place. I'll maintain a file pointer, and once it reaches the max size, reset it to the beginning and continue writing. This way, older logs are continuously overwritten by newer ones, and the file size never exceeds the cap.

Code:

```
import java.io.IOException;
import java.io.RandomAccessFile;

public class RollingLogFile {
    private static final long MAX_FILE_SIZE = 1024; // 1 KB limit
    private final RandomAccessFile logFile;
    private long pointer = 0;

    public RollingLogFile(String path) throws IOException {
        logFile = new RandomAccessFile(path, "rw");
    }

    public synchronized void writeLog(String message) throws IOException {
        byte[] data = (message + System.lineSeparator()).getBytes();

        // Wrap around if next write would exceed the limit
        if (pointer + data.length > MAX_FILE_SIZE) {
            pointer = 0;
        }

        logFile.seek(pointer);
        logFile.write(data);
        pointer = logFile.getFilePointer();
    }

    public void close() throws IOException {
        logFile.close();
    }

    public static void main(String[] args) throws Exception {
        RollingLogFile rolling = new RollingLogFile("audit.log");
        for (int i = 0; i < 50; i++) {
            rolling.writeLog("Event #" + i);
        }
        rolling.close();
```

```
    }
}
```

Reasoning to interviewer:

- RandomAccessFile gives explicit control over where in the file to write.
- I maintain a pointer so I know where the next write will go. Once I hit the cap, I reset to 0 and overwrite from the beginning.
- This ensures the file size is fixed, and the oldest logs are sacrificed for the newest ones.
- Synchronizing writeLog makes it safe if multiple threads log concurrently.

Future scope:

- Add an index or header to track the “current start” so readers can replay logs in correct order.
- Support variable max file size configured at runtime.
- Buffer writes in memory and flush in batches for performance.
- Extend into a multi-file rolling system if audit retention is legally required.

Scenario 24: Thread-safe logging to a single shared file

In a multi-threaded app (like a web server), multiple threads may log at the same time. If writes overlap, the file gets corrupted or log lines get interleaved.

Answer:

The simplest solution is to synchronize the logging method so only one thread writes at a time. This guarantees consistency. For high-throughput systems, I'd use a queue + background writer thread, but synchronization works for moderate workloads.

Code:

```
import java.io.FileWriter;
import java.io.IOException;

public class SynchronizedLogger {
    private final String logFilePath;

    public SynchronizedLogger(String path) {
        this.logFilePath = path;
    }

    // Synchronized ensures only one thread executes this block at a time
    public synchronized void log(String message) {
        try (FileWriter writer = new FileWriter(logFilePath, true)) {
            writer.write(message + System.lineSeparator());
        } catch (IOException e) {
            System.err.println("Error writing log: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        SynchronizedLogger logger = new SynchronizedLogger("threaded.log");

        Runnable task = () -> {
            for (int i = 0; i < 5; i++) {
                logger.log(Thread.currentThread().getName() + " - message " + i);
            }
        };

        Thread t1 = new Thread(task, "Worker-1");
        Thread t2 = new Thread(task, "Worker-2");
        t1.start();
        t2.start();
    }
}
```

Reasoning to interviewer:

- Synchronizing the log method ensures only one thread writes at a time, preventing race conditions.
- Using try-with-resources guarantees the writer is closed after each call, so there’s no resource leak.
- This design works fine when logs are not extremely high volume. For heavy workloads, this could become a bottleneck.

Future scope:

- Use a single long-lived BufferedWriter guarded by synchronization instead of reopening the file each time.
- Implement an async logger: threads enqueue log messages, and one background thread writes them. This scales much better.
- Integrate with existing frameworks (Log4j2, Logback) that already solve concurrency and performance issues.
- Add log levels and formatting to make the logger more production-ready.

Scenario 25: Non-blocking, concurrent reads of large binary files

Say we ingest multiple video or data blobs at once. Blocking each read wastes threads. We want to kick off reads and keep the CPU busy with other work.

Answer:

I'll use AsynchronousFileChannel so each read returns immediately. I can either poll a Future or register a CompletionHandler callback. For multiple files, I'll open one channel per file, schedule reads, and aggregate results via a CountDownLatch or CompletableFuture. I'll also reuse direct ByteBuffers for better I/O throughput.

Code:

```
import java.io.IOException;
import java.nio.ByteBuffer;
import java.nio.channels.AsynchronousFileChannel;
import java.nio.channels.CompletionHandler;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.util.List;
import java.util.concurrent.CountDownLatch;

public class NonBlockingFileReader {

    // Reads the first chunk of each file asynchronously (demo), then signals when
all are done
    public void readFilesAsync(List<String> filePaths) throws InterruptedException {
        CountDownLatch done = new CountDownLatch(filePaths.size());

        for (String fp : filePaths) {
            Path path = Paths.get(fp);
            try {
                AsynchronousFileChannel ch = AsynchronousFileChannel.open(path,
StandardOpenOption.READ);
                ByteBuffer buf = ByteBuffer.allocateDirect(64 * 1024); // 64KB direct
buffer

                ch.read(buf, 0, null, new CompletionHandler<Integer, Void>() {
                    @Override public void completed(Integer bytesRead, Void att) {
                        try (ch) { // auto-close channel here
                            if (bytesRead == -1) {
                                System.out.println(path + " -> EOF");
                            } else {
                                buf.flip();
                                // Process buf[0..bytesRead)
                                System.out.println(path + " -> read " + bytesRead + " bytes
(async)");
                                // ... process bytes ...
```

```
                    }
                } catch (IOException e) {
                    System.err.println(path + " close error: " + e.getMessage());
                } finally {
                    done.countDown();
                }
            }

            @Override public void failed(Throwable exc, Void att) {
                try {
                    System.err.println(path + " read failed: " + exc.getMessage());
                    ch.close();
                } catch (IOException ignored) {}
                done.countDown();
            }
        });
      } catch (IOException e) {
        System.err.println("Open failed for " + path + ": " + e.getMessage());
        done.countDown();
      }
    }

    // Do other useful work here while I/O happens...
    // When we need results:
    done.await();
  }

  public static void main(String[] args) throws InterruptedException {
    new NonBlockingFileReader().readFilesAsync(List.of("a.bin", "b.bin", "c.bin"));
  }
}
```

Reasoning to interviewer:

- AsynchronousFileChannel initiates I/O without tying up a thread; completion comes via callback.
- Using direct buffers helps the OS perform zero-copy transfers where possible.
- A CountDownLatch coordinates “when all reads are done,” while letting the main thread keep working.
- The callback closes the channel and handles errors—no resource leaks.
- Scaling to whole-file reads: schedule multiple chunked reads per file (offset 0, 64KB, 128KB, …) and combine results.

Future scope:

• Switch to CompletableFuture wrappers for cleaner composition and error propagation.
• Implement chunked pipelining (queue chunks to a CPU worker pool) for parse/compute overlap.
• Tune the async channel thread pool via AsynchronousChannelGroup for high concurrency.
• Fall back to FileChannel + thread pool on filesystems where async I/O behaves similarly to blocking.

Scenario 26: Line-by-line CSV validation for very large uploads

Think of a data import endpoint. Files may be GB-scale and messy. We must validate as we stream to avoid OOM and give early feedback.

Answer:

I'll stream with BufferedReader, validate each row as it arrives, and never load the whole file. For correctness, I'll parse with a small quote-aware splitter instead of naïve split(","), which breaks on commas inside quotes. I'll track the line number and report issues immediately (or funnel them to a report sink).

Code:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.List;
import java.util.ArrayList;

public class CSVValidator {

    // Simple CSV line parser supporting quotes and escaped quotes ("")
    private static List<String> parseCsvLine(String line) {
        List<String> fields = new ArrayList<>();
        StringBuilder cur = new StringBuilder();
        boolean inQuotes = false;

        for (int i = 0; i < line.length(); i++) {
            char c = line.charAt(i);
            if (inQuotes) {
                if (c == '"') {
                    // escaped quote?
                    if (i + 1 < line.length() && line.charAt(i + 1) == '"') {
                        cur.append('"');
                        i++;
                    } else {
                        inQuotes = false;
                    }
                } else {
                    cur.append(c);
                }
            } else {
                if (c == ',') {
                    fields.add(cur.toString().trim());
                    cur.setLength(0);
                } else if (c == '"') {
                    inQuotes = true;
                } else {
                    cur.append(c);
```

```
                }
            }
        }
        fields.add(cur.toString().trim());
        return fields;
    }

    // Example rule: expect exactly 5 fields; field[2] must be an integer; field[4] non-
empty
    private static boolean isValid(List<String> cols) {
        if (cols.size() != 5) return false;
        if (cols.get(4).isEmpty()) return false;
        try {
            Integer.parseInt(cols.get(2));
            return true;
        } catch (NumberFormatException nfe) {
            return false;
        }
    }

    public void validateCSV(String filePath) throws IOException {
        try (BufferedReader reader = new BufferedReader(new FileReader(filePath)))
{
            String line;
            long lineNo = 0;
            while ((line = reader.readLine()) != null) {
                lineNo++;
                if (line.isBlank()) continue; // skip empty lines
                List<String> cols = parseCsvLine(line);
                if (!isValid(cols)) {
                    System.out.println("Invalid at line " + lineNo + ": " + line);
                    // optionally: collect to an error file or stop early
                }
                // else: proceed with downstream processing for the valid row
            }
        }
    }

    public static void main(String[] args) throws IOException {
        new CSVValidator().validateCSV("upload.csv");
    }
}
```

Reasoning to interviewer:

- Streaming with BufferedReader keeps memory steady regardless of file size.
- A tiny, quote-aware parser avoids false negatives on lines like "Doe, John",32,"New York, NY",....

• Validation rules are isolated (isValid), so adding checks is easy (types, ranges, required fields).
• We can choose fail-fast (stop on first error) or accumulate all errors for a single user report.

Future scope:

• Swap in a mature library (Apache Commons CSV, OpenCSV) for robust edge cases and headers.
• Parallelize validation by chunking the input and processing with a bounded thread pool (careful with line boundaries).
• Produce a structured error report (CSV of errors) that users can download and fix.
• Support schema files (JSON/YAML) so rules are data-driven instead of hard-coded.

Scenario 27: Resumable file downloads over HTTP (continue after interruption)

We're building a download manager. If a 1.5 GB file stops at 800 MB, we shouldn't start from zero. We'll resume from byte 800 MB, provided the server supports HTTP Range.

Answer:

I'll store progress (e.g., bytes downloaded, ETag/Last-Modified). On resume, I'll send an HTTP Range header like bytes=800000000-. If the server replies 206 Partial Content, I'll append to the existing file using RandomAccessFile.seek(offset). If it returns 200 OK or the validator (ETag/Last-Modified) changed, I'll restart from scratch to avoid corrupt files.

Code:

```
import java.io.*;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.List;
import java.util.Map;

public class ResumableDownloader {

    public void download(String fileUrl, File outFile) throws IOException {
        long already = outFile.exists() ? outFile.length() : 0L;

        HttpURLConnection conn = (HttpURLConnection) new
URL(fileUrl).openConnection();
        if (already > 0) conn.setRequestProperty("Range", "bytes=" + already + "-");

        // Optionally: supply If-Range with a stored ETag or Last-Modified
        // conn.setRequestProperty("If-Range", storedEtagOrLastModified);

        int code = conn.getResponseCode();
        if (already > 0 && code == HttpURLConnection.HTTP_OK) {
            // Server ignored Range; safest is to restart
            already = 0;
        } else if (already > 0 && code == HttpURLConnection.HTTP_PARTIAL) {
            // Good: resume supported
        } else if (code != HttpURLConnection.HTTP_OK) {
            throw new IOException("HTTP error: " + code);
        }

        // Optional: validate Content-Range if resuming
        if (already > 0) {
            String cr = conn.getHeaderField("Content-Range"); // e.g., bytes 800-
.../total
            if (cr == null || !cr.startsWith("bytes " + already + "-")) {
```

```
                // Inconsistent resume; start over
                already = 0;
                conn.disconnect();
                conn = (HttpURLConnection) new URL(fileUrl).openConnection();
                code = conn.getResponseCode();
                if (code != HttpURLConnection.HTTP_OK) throw new
IOException("HTTP error: " + code);
            }
        }

        // Optional: capture ETag/Last-Modified for future validations
        String etag = conn.getHeaderField("ETag");
        String lastMod = conn.getHeaderField("Last-Modified");

        try (InputStream in = new BufferedInputStream(conn.getInputStream());
             RandomAccessFile raf = new RandomAccessFile(outFile, "rw")) {

            if (already > 0) raf.seek(already); else raf.setLength(0);

            byte[] buf = new byte[64 * 1024];
            int n;
            long downloaded = already;
            while ((n = in.read(buf)) != -1) {
                raf.write(buf, 0, n);
                downloaded += n;
                // Optionally: persist progress periodically (downloaded, etag/lastMod)
            }
        } finally {
            conn.disconnect();
        }
    }

    // Helpers to persist/load ETag/Last-Modified across runs can be added as
needed.
}
```

Reasoning to interviewer:

- Range + 206 Partial Content is the standard for resumable HTTP downloads; RandomAccessFile.seek appends at the correct offset.
- I protect against server-side changes using validators (ETag/Last-Modified). If a mismatch occurs, I restart to avoid mixing different versions of the file.
- Using a buffered stream and a 64 KB buffer balances throughput and memory.
- Validating Content-Range ensures the server honored the requested offset; otherwise we'd corrupt the file.

Future scope:

- Parallel segmented downloads (multiple ranges) then stitch segments—guard with ETag checks.
- Add checksum verification (MD5/SHA-256) once the download finishes.
- Auto-retry with exponential backoff on transient network errors.
- Persist download state (offset, ETag, temp file) to survive process restarts cleanly.

Scenario 28: Streaming very large JSON with Jackson (token-by-token parsing)

ETL has to digest multi-GB JSON—loading it into memory is a non-starter. We need a streaming parser that walks tokens and processes records incrementally.

Answer:

I'll use Jackson's low-level JsonParser (streaming API). It emits a stream of tokens (START_OBJECT, FIELD_NAME, VALUE_STRING, etc.). I'll iterate tokens, handle nested structures, and emit each record as soon as it's complete. For large top-level arrays, I'll advance into the array and process each object one at a time.

Code:

```
import com.fasterxml.jackson.core.*;
import com.fasterxml.jackson.databind.ObjectMapper;

import java.io.File;
import java.io.IOException;

public class JSONStreamProcessor {

    private final JsonFactory factory = new JsonFactory();

    // Example input shape:
    // {
    //   "dataset": [
    //     { "id": 1, "name": "A", "metrics": { "views": 10, "ctr": 0.2 } },
    //     { "id": 2, "name": "B", "metrics": { "views": 20, "ctr": 0.15 } }
    //   ]
    // }
    public void processLargeJSON(String filePath) throws IOException {
        try (JsonParser p = factory.createParser(new File(filePath))) {

            // Navigate to the "dataset" array
            if (p.nextToken() != JsonToken.START_OBJECT) return;
            while (p.nextToken() != JsonToken.END_OBJECT) {
                String field = p.getCurrentName();
                if (field == null) continue;

                p.nextToken(); // move to the value of the field
                if ("dataset".equals(field) && p.currentToken() ==
JsonToken.START_ARRAY) {
                    // Iterate items one-by-one, never holding the full array
                    while (p.nextToken() != JsonToken.END_ARRAY) {
                        // p is at START_OBJECT of an item
                        processOneItem(p); // consumes the object
                    }
                } else {
```

```
                // Skip any fields we don't care about
                p.skipChildren();
            }
        }
    }
}

// Consume one object from the parser; extract only what we need.
private void processOneItem(JsonParser p) throws IOException {
    Long id = null;
    String name = null;
    Integer views = null;
    Double ctr = null;

    while (p.nextToken() != JsonToken.END_OBJECT) {
        String fn = p.getCurrentName();
        p.nextToken(); // move to value
        switch (fn) {
            case "id" -> id = p.getLongValue();
            case "name" -> name = p.getValueAsString();
            case "metrics" -> {
                // nested object
                while (p.nextToken() != JsonToken.END_OBJECT) {
                    String m = p.getCurrentName();
                    p.nextToken();
                    if ("views".equals(m)) views = p.getIntValue();
                    else if ("ctr".equals(m)) ctr = p.getDoubleValue();
                    else p.skipChildren();
                }
            }
            default -> p.skipChildren(); // skip unknown nested structures safely
        }
    }

    // Example: stream the record to downstream (DB insert, queue, aggregation,
etc.)
    System.out.printf("Item => id=%s, name=%s, views=%s, ctr=%s%n", id,
name, views, ctr);
}

public static void main(String[] args) throws IOException {
    new JSONStreamProcessor().processLargeJSON("big.json");
}
}
```

Reasoning to interviewer:

- The streaming API keeps memory flat, since we only hold fields for the current object.

• skipChildren() is crucial to ignore subtrees we don’t need without parsing them into objects.

• For a top-level array, we iterate token-by-token—no List accumulation.

• This style is resilient for deeply nested structures and gives full control over partial failure handling (e.g., log and continue on a bad record).

Future scope:

• If you prefer a slightly higher level, use ObjectMapper.readerFor(Type).readValues(File) which returns a MappingIterator<T> to stream POJOs one by one.

• Parallelize downstream processing with a bounded executor; keep the parser single-threaded for simplicity.

• Add schema validation (e.g., JSON Schema) and structured error reporting per record.

• Handle compressed inputs by wrapping the InputStream with GZIPInputStream before feeding the parser.

Scenario 29: Incrementally tailing a remote HTTP file without re-downloading old data

Think “remote tail -f”: a log grows on a server, and your client only wants the freshly appended lines each poll, not the whole file again.

Answer:

I’ll remember how many bytes I’ve already consumed (say in a local state file). On the next poll, I’ll send an HTTP request with a Range: bytes=<last>- header. If the server responds with 206 Partial Content, I’ll stream the new bytes, parse them into lines, and then advance my saved byte offset by the exact number of bytes I actually read. I’ll also store validators (ETag / Last-Modified). If they change, I’ll restart from byte 0 to avoid mixing different file versions.

Code:

```
import java.io.*;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.charset.Charset;
import java.time.Instant;

public class RemoteTail {

    private final String fileUrl;
    private final Charset charset;
    private long lastBytePos = 0;        // persisted across runs
    private String lastEtag = null;      // optional validator
    private String lastModified = null;  // optional validator

    public RemoteTail(String fileUrl, Charset charset) {
        this.fileUrl = fileUrl;
        this.charset = charset;
    }

    public void readNewLines() throws IOException {
        HttpURLConnection conn = (HttpURLConnection) new
URL(fileUrl).openConnection();
        if (lastBytePos > 0) {
            conn.setRequestProperty("Range", "bytes=" + lastBytePos + "-");
            if (lastEtag != null) conn.setRequestProperty("If-Range", lastEtag);
            else if (lastModified != null) conn.setRequestProperty("If-Range",
lastModified);
        }

        int code = conn.getResponseCode();
        if (code == HttpURLConnection.HTTP_OK && lastBytePos > 0) {
            // Server didn’t honor Range; safest is to reset and start over
            lastBytePos = 0;
```

```
        } else if (code != HttpURLConnection.HTTP_OK && code !=
HttpURLConnection.HTTP_PARTIAL) {
            throw new IOException("HTTP error: " + code);
        }

        // capture validators for next time
        lastEtag = conn.getHeaderField("ETag");
        lastModified = conn.getHeaderField("Last-Modified");

        long newlyReadBytes = 0;
        try (InputStream rawIn = new BufferedInputStream(conn.getInputStream())) {
            // We'll decode text but count raw bytes read to advance lastBytePos
correctly.
            ByteArrayOutputStream buf = new ByteArrayOutputStream(64 * 1024);
            byte[] tmp = new byte[64 * 1024];
            int n;
            while ((n = rawIn.read(tmp)) != -1) {
                buf.write(tmp, 0, n);
                newlyReadBytes += n;
            }
            // Process new bytes into lines (simple approach; for perfect multibyte
safety, use a CharsetDecoder)
            String text = buf.toString(charset);
            for (String line : text.split("\\R", -1)) {
                if (!line.isEmpty()) {
                    System.out.println(line);
                }
            }
        } finally {
            conn.disconnect();
        }

        // Move the window forward by the exact bytes fetched
        lastBytePos += newlyReadBytes;
        // persist lastBytePos/ETag/Last-Modified to disk so the next run resumes
correctly
        // (omitted here for brevity)
    }

    // Demo
    public static void main(String[] args) throws Exception {
        RemoteTail tail = new RemoteTail("https://example.com/logs/app.log",
Charset.forName("UTF-8"));
        tail.readNewLines(); // call periodically (cron/scheduler)
        // Next call will fetch only the bytes appended since the previous run
    }
}
```

Reasoning to interviewer:

- Using Range plus validators (ETag/Last-Modified) is the standard way to fetch only new bytes.
- I count raw bytes from the InputStream to advance the offset correctly (line length in characters ≠ bytes, especially with UTF-8).
- If the server ignores Range (returns 200), or the validators change, I reset to avoid file corruption/misalalignment.
- Splitting on \R keeps line handling cross-platform. For perfect multibyte boundary handling across polls, I'd switch to a CharsetDecoder and carry over incomplete code points between polls.

Future scope:

- Persist state in a sidecar JSON (offset, etag, lastModified, timestamp) and handle clock skew.
- Handle HTTP 416 (Requested Range Not Satisfiable) gracefully by resetting.
- Add backoff/retry and proxy/TLS options.
- Stream-process lines as they arrive instead of buffering the whole response if logs are huge.

Scenario 30: Multi-threaded downloader that records progress and resumes after interruptions

Think a small IDM: multiple files in parallel, each keeping its own resume state, and periodically checkpointing progress so you don't lose work on a crash.

Answer:

I'll model each download as a Runnable that:

- Loads prior progress (bytes already on disk) from a sidecar progress file and the partial target file's length.
- Sends Range to resume from that offset; validates with ETag/Last-Modified.
- Writes via RandomAccessFile.seek(offset).
- Every N bytes (e.g., 1 MB) it saves the current byte offset and validators.

  A fixed thread pool runs many such tasks concurrently.

Code:

```
import java.io.*;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.file.*;
import java.util.List;
import java.util.concurrent.*;

public class FileDownloader implements Runnable {
    private static final int BUFFER = 64 * 1024;
    private static final long CHECKPOINT_EVERY = 1L * 1024 * 1024; // 1 MB

    private final String url;
    private final Path outPath;
    private final Path progressPath;

    public FileDownloader(String url, String outputFilePath) {
        this.url = url;
        this.outPath = Paths.get(outputFilePath);
        this.progressPath = Paths.get(outputFilePath + ".progress");
    }

    record Progress(long offset, String etag, String lastModified) implements
Serializable {}

    private Progress loadProgress() {
        if (!Files.exists(progressPath)) return new Progress(0L, null, null);
        try (ObjectInputStream ois = new
ObjectInputStream(Files.newInputStream(progressPath))) {
```

```
            return (Progress) ois.readObject();
        } catch (Exception e) {
            return new Progress(0L, null, null); // fallback on error
        }
    }

    private void saveProgress(Progress p) {
        try (ObjectOutputStream oos = new
ObjectOutputStream(Files.newOutputStream(progressPath,
                StandardOpenOption.CREATE,
StandardOpenOption.TRUNCATE_EXISTING))) {
            oos.writeObject(p);
        } catch (IOException ignored) {}
    }

    @Override
    public void run() {
        Progress p = loadProgress();
        long already = p.offset();

        try {
            if (Files.exists(outPath)) {
                long currentLen = Files.size(outPath);
                // trust the larger of stored offset vs file length
                already = Math.max(already, currentLen);
            }

            HttpURLConnection conn = (HttpURLConnection) new
URL(url).openConnection();
            if (already > 0) {
                conn.setRequestProperty("Range", "bytes=" + already + "-");
                if (p.etag() != null) conn.setRequestProperty("If-Range", p.etag());
                else if (p.lastModified() != null) conn.setRequestProperty("If-Range",
p.lastModified());
            }

            int code = conn.getResponseCode();
            if (already > 0 && code == HttpURLConnection.HTTP_OK) {
                // server ignored range; restart safely
                already = 0;
            } else if (code != HttpURLConnection.HTTP_OK && code !=
HttpURLConnection.HTTP_PARTIAL) {
                throw new IOException("HTTP error: " + code);
            }

            // Capture validators
            String etag = conn.getHeaderField("ETag");
            String lastMod = conn.getHeaderField("Last-Modified");

            try (InputStream in = new BufferedInputStream(conn.getInputStream());
```

```
             RandomAccessFile raf = new RandomAccessFile(outPath.toFile(),
"rw")) {

            if (already > 0) raf.seek(already); else raf.setLength(0);

            byte[] buf = new byte[BUFFER];
            int n;
            long downloaded = already;
            long nextCheckpoint = downloaded + CHECKPOINT_EVERY;

            while ((n = in.read(buf)) != -1) {
                raf.write(buf, 0, n);
                downloaded += n;
                if (downloaded >= nextCheckpoint) {
                    saveProgress(new Progress(downloaded, etag, lastMod));
                    nextCheckpoint += CHECKPOINT_EVERY;
                }
            }

            // final checkpoint and clean up progress on success
            saveProgress(new Progress(downloaded, etag, lastMod));
            // optionally delete the .progress file to signal completion
            // Files.deleteIfExists(progressPath);
        } finally {
            conn.disconnect();
        }

    } catch (IOException e) {
        System.err.println("Download failed for " + url + ": " + e.getMessage());
    }
  }

  // Demo runner
  public static void main(String[] args) throws InterruptedException {
    List<FileDownloader> tasks = List.of(
        new FileDownloader("https://example.com/file1.bin", "file1.bin"),
        new FileDownloader("https://example.com/file2.bin", "file2.bin"),
        new FileDownloader("https://example.com/file3.bin", "file3.bin")
    );

    ExecutorService pool = Executors.newFixedThreadPool(3);
    tasks.forEach(pool::submit);
    pool.shutdown();
    pool.awaitTermination(1, TimeUnit.HOURS);
  }
}
```

Reasoning to interviewer:

- RandomAccessFile.seek(offset) + Range requests give clean resume semantics.
- Sidecar progress files keep state durable across crashes; storing ETag/Last-Modified avoids mixing different server versions.
- Periodic checkpointing every 1 MB balances durability and filesystem churn.
- Using a thread pool controls parallelism and keeps the code scalable.

Future scope:

- Validate Content-Range to ensure the server honored the resume offset.
- Add checksums (SHA-256) and auto-retry with exponential backoff.
- Support parallel segmented downloads per file and merge at the end.
- Persist progress in a central store (SQLite/Redis) if multiple JVMs may resume the same downloads.

Scenario 31: In-memory cache that periodically persists to disk and restores on startup

We want a fast in-memory lookup (for hot configs, feature flags, SKU metadata, etc.) but also durability across restarts. Think of it like a tiny write-through cache with timed snapshots.

Answer:

I'll keep the cache as a thread-safe map. On startup, I'll try to hydrate it by deserializing a snapshot file. During runtime, a scheduled task will periodically serialize the map to disk. To avoid partial/corrupt snapshots, I'll write to a temp file and atomically move it over the old one. For concurrent writes, I'll snapshot an immutable copy before serializing so we never block readers for long.

Code:

```
import java.io.*;
import java.nio.file.*;
import java.util.Map;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicReference;

public class PersistentCache implements Closeable, Serializable {
    private static final long serialVersionUID = 1L;

    private final Path snapshotPath;
    private final ScheduledExecutorService scheduler =
Executors.newSingleThreadScheduledExecutor();

    // Thread-safe in-memory store
    private final ConcurrentHashMap<String, String> map = new
ConcurrentHashMap<>();

    public PersistentCache(Path snapshotPath) {
        this.snapshotPath = snapshotPath;
        loadFromDiskIfPresent();
        // snapshot every 60 seconds (tune as needed)
        scheduler.scheduleAtFixedRate(this::saveToDiskSafe, 60, 60,
TimeUnit.SECONDS);
    }

    public void put(String k, String v) { map.put(k, v); }

    public String get(String k) { return map.get(k); }

    public void putAll(Map<String, String> entries) { map.putAll(entries); }

    // Manual flush
    public void flush() { saveToDiskSafe(); }
```

```
    private void loadFromDiskIfPresent() {
        if (!Files.exists(snapshotPath)) return;
        try (ObjectInputStream in = new
ObjectInputStream(Files.newInputStream(snapshotPath))) {
            @SuppressWarnings("unchecked")
            Map<String, String> disk = (Map<String, String>) in.readObject();
            map.clear();
            map.putAll(disk);
        } catch (Exception e) {
            System.out.println("Cache restore skipped: " + e.getMessage());
        }
    }

    private void saveToDiskSafe() {
        // Snapshot a stable view so serialization isn't blocked by live mutations
        AtomicReference<Map<String,String>> snapRef = new
AtomicReference<>(Map.copyOf(map));
        Path tmp = snapshotPath.resolveSibling(snapshotPath.getFileName() +
".tmp");
        try (ObjectOutputStream out = new
ObjectOutputStream(Files.newOutputStream(tmp,
                StandardOpenOption.CREATE,
StandardOpenOption.TRUNCATE_EXISTING))) {
            out.writeObject(snapRef.get());
            out.flush();
            // Atomic replace to avoid partial writes
            Files.move(tmp, snapshotPath,
StandardCopyOption.REPLACE_EXISTING,
StandardCopyOption.ATOMIC_MOVE);
        } catch (IOException io) {
            try { Files.deleteIfExists(tmp); } catch (IOException ignored) {}
            System.err.println("Cache snapshot failed: " + io.getMessage());
        }
    }

    @Override public void close() {
        scheduler.shutdown();
        saveToDiskSafe();
    }

    // tiny demo
    public static void main(String[] args) throws Exception {
        Path path = Paths.get("cache.snapshot");
        try (PersistentCache cache = new PersistentCache(path)) {
            cache.put("feature.payments.v2", "enabled");
            cache.put("sku:123.price", "499");
            cache.flush(); // ensure persisted now
        }
        // next boot will restore from cache.snapshot
```

```
    }
}
```

Reasoning to interviewer:

- ConcurrentHashMap gives lock-free reads and fine-grained writes.
- Atomic temp-file → move prevents torn snapshots if the process crashes mid-write.
- Copying to an immutable Map.copyOf(...) before serialization avoids holding locks while writing to disk.
- Scheduled snapshots balance durability and I/O; we can also expose an explicit flush() on important updates.

Future scope:

- Add TTL/eviction (size-based LRU) and only persist the hot subset.
- Version the snapshot format; migrate gracefully when schema changes.
- Encrypt snapshots at rest if keys/values are sensitive.
- Switch to a write-ahead log (append) plus compaction for frequent updates.
- Use JSON/CBOR/Protostuff for language-agnostic snapshots.

Scenario 32: Encrypting sensitive files at rest and decrypting on read

We're storing confidential data (PII, tokens). If the disk is compromised, raw files must be unreadable. We'll use authenticated encryption so tampering is detectable.

Answer:

I'll use AES in GCM mode (AES/GCM/NoPadding) with a fresh random IV per file. The IV is stored alongside the ciphertext. On write: generate IV, init cipher in ENCRYPT mode, stream bytes through CipherOutputStream. On read: fetch IV, init cipher in DECRYPT mode, stream through CipherInputStream. The key comes from a SecretKeySpec (in production it should come from a KMS or keystore, not hard-coded).

Code (encrypt + decrypt with AES-GCM):

```
import javax.crypto.*;
import javax.crypto.spec.GCMParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.io.*;
import java.nio.file.*;
import java.security.SecureRandom;

public class FileCrypto {

    // 256-bit key preferred; ensure your JCE policy allows it (most modern JDKs
do).
    private final SecretKey key;
    private static final String TRANSFORM = "AES/GCM/NoPadding";
    private static final int IV_BYTES = 12;        // 96-bit IV recommended for GCM
    private static final int TAG_BITS = 128;       // 128-bit auth tag

    public FileCrypto(byte[] rawKey) {
        this.key = new SecretKeySpec(rawKey, "AES");
    }

    public void encrypt(Path input, Path output) throws Exception {
        byte[] iv = new byte[IV_BYTES];
        new SecureRandom().nextBytes(iv);

        Cipher cipher = Cipher.getInstance(TRANSFORM);
        cipher.init(Cipher.ENCRYPT_MODE, key, new
GCMParameterSpec(TAG_BITS, iv));

        // Write IV prefix, then ciphertext
        try (OutputStream fout = Files.newOutputStream(output,
StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING);
             BufferedOutputStream bout = new BufferedOutputStream(fout);
             ) {
```

```
            bout.write(iv); // store IV in clear
            try (CipherOutputStream cout = new CipherOutputStream(bout, cipher);
                 InputStream in = new
BufferedInputStream(Files.newInputStream(input))) {
                in.transferTo(cout);
            }
        }
    }

    public void decrypt(Path input, Path output) throws Exception {
        try (InputStream fin = new
BufferedInputStream(Files.newInputStream(input))) {
            byte[] iv = fin.readNBytes(IV_BYTES);
            if (iv.length != IV_BYTES) throw new IOException("Invalid encrypted file
header");

            Cipher cipher = Cipher.getInstance(TRANSFORM);
            cipher.init(Cipher.DECRYPT_MODE, key, new
GCMParameterSpec(TAG_BITS, iv));

            try (CipherInputStream cin = new CipherInputStream(fin, cipher);
                 OutputStream out = new
BufferedOutputStream(Files.newOutputStream(output,
StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING))) {
                cin.transferTo(out);
            }
        }
    }

    // demo only; never hard-code keys in production
    public static void main(String[] args) throws Exception {
        byte[] keyBytes = new byte[32]; // 256-bit
        new SecureRandom().nextBytes(keyBytes);

        FileCrypto crypto = new FileCrypto(keyBytes);
        Path plain = Paths.get("secret.txt");
        Path enc = Paths.get("secret.enc");
        Path dec = Paths.get("secret.dec.txt");

        Files.writeString(plain, "Top secret: launch at 0900.");

        crypto.encrypt(plain, enc);
        crypto.decrypt(enc, dec);
    }
}
```

Reasoning to interviewer:

• AES-GCM provides confidentiality and integrity; if ciphertext is modified, decryption fails with AEAD auth errors.
• Fresh random IV per file is mandatory; we persist it as a header before ciphertext.
• Streaming via CipherOutputStream/CipherInputStream handles arbitrarily large files without loading them into memory.
• Keys must be managed securely (keystore/HSM/KMS); the demo uses a random in-memory key only for illustration.

Future scope:

• Integrate with a keystore or cloud KMS; rotate keys and re-encrypt snapshots.
• Add associated data (AAD) to bind metadata (e.g., filename, user id) to the ciphertext.
• Add file version headers and integrity checksums of plaintext metadata.
• Parallelize large-file encryption by chunking with unique nonces and per-chunk tags, then verify all tags.

Scenario 33: Serialize objects for clients while excluding sensitive fields

We're sending POJOs over the wire (say, user profiles), but things like passwords/tokens must never leave the server. The payload should include only safe fields.

Answer:

I'd first mark obviously sensitive members transient so default serialization skips them. For finer control (masking, derived fields, conditional logic), I'll customize writeObject/readObject. If the contract is external and versioned, I prefer the Serialization Proxy pattern or an explicit DTO to fully control what leaves the JVM.

Code:

```
import java.io.*;

// DTO sent to clients; sensitiveData is excluded
class ClientUpdate implements Serializable {
    private static final long serialVersionUID = 1L;

    private final String username;
    private transient String sensitiveData; // skipped by default serialization

    public ClientUpdate(String username, String sensitiveData) {
        this.username = username;
        this.sensitiveData = sensitiveData;
    }

    // Optional: custom serialization (e.g., mask, compute derived fields)
    private void writeObject(ObjectOutputStream out) throws IOException {
        out.defaultWriteObject(); // writes username
        // Do NOT write sensitiveData
        // If you want to indicate presence: out.writeBoolean(sensitiveData != null);
    }

    private void readObject(ObjectInputStream in) throws IOException,
ClassNotFoundException {
        in.defaultReadObject();
        // Rehydrate transient fields if you have a secure source; otherwise keep null
        this.sensitiveData = null;
    }

    @Override public String toString() {
        return "ClientUpdate{username='" + username + "', sensitiveData=" +
(sensitiveData == null ? "null" : "***") + "}";
    }
}
```

```
// Safer, explicit wire model using a proxy/DTO
final class ClientUpdateView implements Serializable {
    private static final long serialVersionUID = 1L;
    private final String username;
    ClientUpdateView(String username) { this.username = username; }
    public static ClientUpdateView from(ClientUpdate cu) { return new
ClientUpdateView(cu.toString()); }
}
```

Reasoning to interviewer:

- transient is the minimal, safe guardrail—sensitive fields never hit the stream.
- Overriding writeObject/readObject gives surgical control (masking, omitting, versioning).
- For long-term stability or cross-service contracts, a DTO/serialization proxy avoids accidental leaks when someone adds a new field later.

Future scope:

- Use Externalizable for full manual control if the default mechanism is too implicit.
- Prefer JSON/Protobuf DTOs for language-agnostic clients and explicit schemas.
- Add field-level encryption if a subset must travel but remain protected.
- Add class-level serialVersionUID and evolve with backward-compatible readers.

Scenario 34: Random access to large image files for zoom/pan without loading all bytes

Think of a viewer that jumps to arbitrary tiles of a gigapixel image. We need fast, partial reads by byte region.

Answer:

I'll use RandomAccessFile + FileChannel.map to memory-map only the section I need. With MappedByteBuffer, the OS pages that slice lazily, so we avoid reading the entire file. If dealing with structured formats (JPEG/PNG/TIFF), I'll pair this with format-aware region decoders where possible.

Code:

```
import java.io.RandomAccessFile;
import java.nio.MappedByteBuffer;
import java.nio.channels.FileChannel;

public class ImageSectionReader {

    // Reads a raw byte window [position, position+size)
    public byte[] readImageSection(String filePath, long position, int size) throws
Exception {
        try (RandomAccessFile raf = new RandomAccessFile(filePath, "r");
             FileChannel channel = raf.getChannel()) {

            // guard against overflows
            long end = Math.min(channel.size(), position + size);
            int actual = (int)(end - position);
            if (actual <= 0) return new byte[0];

            // map only the slice we need (<= 2GB per mapping)
            MappedByteBuffer mbb =
channel.map(FileChannel.MapMode.READ_ONLY, position, actual);
            byte[] data = new byte[actual];
            mbb.get(data);
            return data;
        }
    }
}
```

Reasoning to interviewer:

- Memory mapping delegates paging to the OS; we get O(1) random access without explicit read loops.
- We cap the mapping length to avoid the int capacity limit on MappedByteBuffer.

- For compressed image formats, bytes ≠ pixels; random byte windows may not align with decode units. When available, Java Image I/O readers (e.g., ImageReader.readRaster(region)) or TIFF readers with tile/strip access are better to fetch decoded regions.

Future scope:

- Use format-aware region decoders (ImageIO plugins, TIFF libraries) to fetch pixel tiles directly.
- Implement a tile cache (LRU) keyed by zoom level and region to avoid re-decoding.
- For massive files, map once and reuse the channel; slice via buffer positions to reduce mapping churn.
- Consider FileChannel.read(ByteBuffer, position) if mapping isn't desirable (e.g., containerized memory constraints).

Scenario 35: Streaming audio in small chunks to minimize memory

We don't want to load a full MP3/WAV into memory. Like a real streaming server, we'll read fixed-size byte blocks and push them over the wire continuously.

Answer:

I'll open the file as a byte stream, wrap it in a BufferedInputStream, and loop over a fixed-size buffer (e.g., 32 KB). Each chunk is forwarded to the network/socket. This keeps memory flat and latency low. If we ever need throttling (simulate bitrate) or seeking, we can add it around the same loop.

Code:

```
import java.io.BufferedInputStream;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.OutputStream;
import java.net.Socket;

public class AudioStreamer {

    // Streams audio file to a client socket in fixed-size chunks
    public void streamAudio(String filePath, Socket client) throws IOException {
        try (BufferedInputStream in = new BufferedInputStream(new
FileInputStream(filePath));
             OutputStream out = client.getOutputStream()) {

            byte[] buffer = new byte[32 * 1024]; // 32 KB chunks
            int n;
            while ((n = in.read(buffer)) != -1) {
                out.write(buffer, 0, n);
                // Optionally: out.flush() for low-latency live streaming
                // Optionally: sleep to throttle to target bitrate
            }
        }
    }
}
```

Reasoning to interviewer:

- Buffered streaming reduces system calls and avoids loading the whole file.
- A 32 KB buffer balances throughput and latency; we can tune it.
- Writing directly to the socket keeps a steady flow; no large memory spikes.
- This is format-agnostic: MP3/AAC/WAV are just bytes at this layer. Higher layers (codecs/headers) can run alongside if needed.

Future scope:

- Add HTTP/Chunked transfer or WebSocket framing for browser clients.
- Support seek by jumping file position (FileChannel.position(long)).
- Implement rate limiting to match target bitrate and avoid client buffer overflow.
- For live transcoding, pipe bytes through a codec process before sending.

Scenario 36: Processing a huge JSON file without loading it entirely

Think analytics/ETL pulling multi-GB JSON. We'll parse records one at a time to keep memory constant.

Answer:

I'll use Jackson's streaming API (JsonFactory + JsonParser). It emits tokens like START_ARRAY, FIELD_NAME, VALUE_STRING. I'll walk tokens, handle just the fields I need, and skip subtrees I don't. If the top level is a big array, I'll iterate each element object-by-object.

Code:

```
import com.fasterxml.jackson.core.*;
import java.io.File;
import java.io.IOException;

public class JSONStreamReader {

    // Example: top-level array of objects: [ { "name": "...", "age": 30, ... }, ... ]
    public void parseLargeJSON(String filePath) throws IOException {
        JsonFactory factory = new JsonFactory();
        try (JsonParser p = factory.createParser(new File(filePath))) {

            // Move to first token
            JsonToken t = p.nextToken();
            if (t == JsonToken.START_ARRAY) {
                // Stream array elements one-by-one
                while (p.nextToken() != JsonToken.END_ARRAY) {
                    readOneObject(p); // consumes one object
                }
            } else if (t == JsonToken.START_OBJECT) {
                // Object with nested arrays/fields; adapt as needed
                while (p.nextToken() != JsonToken.END_OBJECT) {
                    String field = p.getCurrentName();
                    p.nextToken(); // move to value
                    if ("dataset".equals(field) && p.currentToken() ==
JsonToken.START_ARRAY) {
                        while (p.nextToken() != JsonToken.END_ARRAY) {
                            readOneObject(p);
                        }
                    } else {
                        p.skipChildren(); // ignore unneeded trees
                    }
                }
            }
        }
    }
```

```
    // Consume exactly one JSON object at the current parser position
    private void readOneObject(JsonParser p) throws IOException {
        String name = null;
        Integer age = null;

        while (p.nextToken() != JsonToken.END_OBJECT) {
            String fn = p.getCurrentName();
            p.nextToken(); // move to the value for this field
            switch (fn) {
                case "name" -> name = p.getValueAsString();
                case "age"  -> age  = p.getValueAsInt();
                default     -> p.skipChildren(); // safely skip nested/unknown structures
            }
        }

        // Process record (DB insert, aggregation, etc.)
        if (name != null && age != null) {
            System.out.println("User: " + name + ", age=" + age);
        }
    }
}
```

Reasoning to interviewer:

- Streaming parser keeps memory usage flat—only current record is in scope.
- skipChildren() is key to efficiently ignore unneeded nested data.
- Works with arbitrarily large inputs (no List accumulation).
- Token-by-token control makes it easy to validate fields and handle partial failures (log bad rows, continue).

Future scope:

- Use ObjectMapper.readerFor(Type).readValues(...) to stream POJOs via MappingIterator<T>.
- Add schema validation and error reporting per record.
- Decompress on the fly by wrapping the input with GZIPInputStream.
- Parallelize downstream handling with a bounded thread pool while keeping parsing single-threaded.

Scenario 37: Watching a directory for create/modify/delete events in real-time

We want something like inotify on Linux, but in Java, so that whenever files are created, updated, or deleted, our app reacts immediately (e.g., for log collectors, config reloading, or sync tools).

Answer:

I'll use Java NIO's WatchService. You register a Path with event kinds (ENTRY_CREATE, ENTRY_MODIFY, ENTRY_DELETE). The WatchService.take() blocks until an event occurs. Each WatchKey carries a batch of events; after handling them, you reset the key to continue watching. For multiple directories, register each Path. For recursive subdirectories, you need to register children individually.

Code:

```
import java.io.IOException;
import java.nio.file.*;

public class DirectoryWatcher {

    public void watchDirectory(String directoryPath) throws IOException,
InterruptedException {
        Path dir = Paths.get(directoryPath);
        WatchService watcher = FileSystems.getDefault().newWatchService();

        dir.register(watcher,
                StandardWatchEventKinds.ENTRY_CREATE,
                StandardWatchEventKinds.ENTRY_MODIFY,
                StandardWatchEventKinds.ENTRY_DELETE);

        System.out.println("Watching: " + dir.toAbsolutePath());

        while (true) {
            WatchKey key = watcher.take(); // blocks until events
            for (WatchEvent<?> event : key.pollEvents()) {
                WatchEvent.Kind<?> kind = event.kind();
                Path affected = (Path) event.context();
                System.out.printf("[%s] %s%n", kind.name(), affected);
            }
            boolean valid = key.reset();
            if (!valid) {
                System.out.println("WatchKey no longer valid; stopping watch.");
                break;
            }
        }
    }
```

```
    public static void main(String[] args) throws Exception {
        new DirectoryWatcher().watchDirectory("testdir");
    }
}
```

Reasoning to interviewer:

- WatchService uses efficient OS-specific hooks (inotify/ReadDirectoryChangesW/FSEvents).
- Each WatchEvent gives relative filename; combine with the base Path for absolute.
- Must call reset() to re-arm the key; if reset fails, the directory is no longer accessible.
- This only watches the registered folder, not subfolders recursively (you'd need to register those separately).

Future scope:

- Add recursive registration with a helper to watch all subdirs.
- Handle OVERFLOW event kind to detect missed events if the queue overflows.
- Wrap this loop in an executor and dispatch events to a handler interface.
- Add debounce logic (e.g., batch multiple MODIFY events for the same file).

Scenario 38: Compress multiple files into a single ZIP archive

Imagine you have 10 text reports and want to bundle them as one .zip download for users, while preserving names.

Answer:

I'll use ZipOutputStream. For each file, I'll create a ZipEntry with its original filename (not the full OS path—just the name or relative path), then stream the file's content into the zip. Once all entries are written, close the ZipOutputStream. This works for any file type, not just text.

Code:

```
import java.io.*;
import java.nio.file.*;
import java.util.zip.ZipEntry;
import java.util.zip.ZipOutputStream;

public class ZipCompressor {

    public void compressFiles(String[] files, String zipFilePath) throws IOException {
        try (ZipOutputStream zipOut = new ZipOutputStream(new
FileOutputStream(zipFilePath))) {
            for (String file : files) {
                Path path = Paths.get(file);
                String entryName = path.getFileName().toString(); // keep just filename
                try (FileInputStream fis = new FileInputStream(path.toFile())) {
                    zipOut.putNextEntry(new ZipEntry(entryName));

                    byte[] buffer = new byte[4096];
                    int len;
                    while ((len = fis.read(buffer)) > 0) {
                        zipOut.write(buffer, 0, len);
                    }

                    zipOut.closeEntry();
                }
            }
        }
        System.out.println("Created ZIP at: " + zipFilePath);
    }

    public static void main(String[] args) throws Exception {
        String[] inputFiles = {"report1.txt", "report2.txt", "report3.txt"};
        new ZipCompressor().compressFiles(inputFiles, "archive.zip");
    }
}
```

Reasoning to interviewer:

- ZipOutputStream handles compression automatically (DEFLATE).
- ZipEntry name defines how the file is seen inside the archive. Using getFileName() ensures we don’t leak absolute paths.
- Buffering (4 KB or larger) improves efficiency compared to single-byte reads.
- Each file is streamed in, so memory usage stays small even for large files.

Future scope:

- Add subdirectory support by setting ZipEntry with relative path strings (e.g., docs/report1.txt).
- Set compression level with zipOut.setLevel(Deflater.BEST_COMPRESSION).
- Add CRC checksums or comments to entries.
- For massive archives, stream output to network directly (HTTP response) instead of creating a temp .zip file.

Scenario 39: Buffered logging that flushes in batches on a schedule

In a chat app, hammering the disk for every single message is wasteful. It's better to buffer logs in memory and flush them periodically (or when the buffer gets large), so we trade a tiny delay for big I/O savings.

Answer:

I'll keep an in-memory queue for incoming log lines and a scheduler that wakes up every few seconds to flush. During flush, I'll atomically swap the "active" buffer with an empty one so writers are never blocked by disk I/O. I'll also add a max buffer size (emergency flush) and a shutdown hook to persist any leftovers on exit.

Code:

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicReference;

public class BufferedLogger implements AutoCloseable {
    private final AtomicReference<List<String>> active = new
AtomicReference<>(new ArrayList<>());
    private final ScheduledExecutorService scheduler =
Executors.newSingleThreadScheduledExecutor();
    private final String filePath;
    private final int maxBuffer;          // e.g., 5_000 lines
    private final long flushIntervalMs;   // e.g., 5_000 ms

    public BufferedLogger(String filePath, int maxBuffer, long flushIntervalMs) {
        this.filePath = filePath;
        this.maxBuffer = maxBuffer;
        this.flushIntervalMs = flushIntervalMs;
        start();
        Runtime.getRuntime().addShutdownHook(new Thread(this::safeFlush));
    }

    public void log(String message) {
        List<String> buf = active.get();
        synchronized (buf) {
            buf.add(Instant.now() + " " + message);
            if (buf.size() >= maxBuffer) {
                flushNow(); // emergency flush when buffer is large
            }
        }
```

```
    }

    private void start() {
        scheduler.scheduleAtFixedRate(this::safeFlush, flushIntervalMs,
flushIntervalMs, TimeUnit.MILLISECONDS);
    }

    // Swap buffers quickly, then write the snapshot outside the lock
    private void flushNow() {
        List<String> oldBuf = active.getAndSet(new ArrayList<>());
        writeAll(oldBuf);
    }

    private void safeFlush() {
        try { flushNow(); } catch (Exception e) { /* log somewhere else */ }
    }

    private void writeAll(List<String> lines) {
        if (lines.isEmpty()) return;
        // synchronize only on file writing, not on the live buffer
        try (BufferedWriter out = new BufferedWriter(new FileWriter(filePath, true))) {
            for (String s : lines) {
                out.write(s);
                out.newLine();
            }
        } catch (IOException e) {
            // Optionally push lines back or route to a fallback logger
            e.printStackTrace();
        } finally {
            lines.clear(); // allow GC
        }
    }

    @Override
    public void close() {
        scheduler.shutdown();
        safeFlush();
    }

    // Demo
    public static void main(String[] args) throws Exception {
        try (BufferedLogger logger = new BufferedLogger("chat.log", 5_000, 5_000)) {
            for (int i = 0; i < 20_000; i++) logger.log("msg #" + i);
        }
    }
}
```

Reasoning to interviewer:

• Double-buffering via AtomicReference means log producers are blocked only long enough to add to the current list; file I/O happens on a detached snapshot.

• Time-based plus size-based flushing keeps latency bounded and prevents unbounded growth.

• A shutdown hook ensures we don’t lose the last batch on JVM exit.

• BufferedWriter with append mode reduces syscalls and preserves existing logs.

Future scope:

• Add backpressure (reject/queue) when the buffer grows too fast.

• Batch by time, size, or both; expose metrics (dropped lines, queue depth).

• Switch to an async ring buffer (e.g., Disruptor) for very high throughput.

• Log rotation + compression of old files.

Scenario 40: Splitting a large file into fixed-size chunks

For multi-part uploads, we need deterministic slices (say 8 MB each) written out as .part0, .part1, … without loading the whole file.

Answer:

I'll use FileChannel and iterate offsets in fixed-size steps. Two efficient approaches:

1. transferTo to stream bytes directly from source channel to each part's channel (fast, minimal copying), or
2. map a window with FileChannel.map and write it out. I'll show transferTo because it handles very large files and avoids mapping limits.

Code:

```
import java.io.IOException;
import java.nio.channels.FileChannel;
import java.nio.file.*;
import java.util.EnumSet;

public class FileSplitter {

    // chunkSizeBytes is a long: e.g., 8L * 1024 * 1024 for 8MB
    public void split(String filePath, long chunkSizeBytes) throws IOException {
        Path src = Paths.get(filePath);
        long size;
        try (FileChannel in = FileChannel.open(src,
EnumSet.of(StandardOpenOption.READ))) {
            size = in.size();
            long offset = 0L;
            int part = 0;

            while (offset < size) {
                long remaining = size - offset;
                long bytesThisPart = Math.min(chunkSizeBytes, remaining);
                Path out = Paths.get(filePath + ".part" + part);

                try (FileChannel outCh = FileChannel.open(out,
                        EnumSet.of(StandardOpenOption.CREATE,
StandardOpenOption.TRUNCATE_EXISTING, StandardOpenOption.WRITE))) {
                    long transferred = 0L;
                    while (transferred < bytesThisPart) {
                        long n = in.transferTo(offset + transferred, bytesThisPart -
transferred, outCh);
                        if (n <= 0) break; // defensive: should not happen under normal
conditions
                        transferred += n;
```

```
                }
            }

            offset += bytesThisPart;
            part++;
        }

        System.out.println("File successfully split into " + part + " parts.");
    }
}

// Demo
public static void main(String[] args) throws IOException {
    new FileSplitter().split("bigfile.bin", 8L * 1024 * 1024); // 8 MB parts
}
}
```

Reasoning to interviewer:

- transferTo is typically zero-copy (kernel-to-kernel path) on many platforms, which is faster than read-then-write loops.
- Using long for sizes/offsets avoids int overflow and supports multi-GB files.
- Each part is produced independently; naming keeps natural order.
- This approach doesn’t pin huge memory and is resilient for very large inputs.

Future scope:

- Parallelize uploads: start uploading each part as soon as it’s created.
- Add a manifest with part checksums and sizes for integrity verification.
- Support resuming by skipping parts that already exist with matching checksum.
- Switch to memory-mapped windows (map) if random access into chunks is needed later (e.g., reassembly with seeks).